

Kritische Schutzmassnahmen für Netzwerke

Linus Espach
Wintersemester 2025



Zielsetzung

Neben dem SASE Ansatz gibt es noch weitere wichtige Schutzkonzepte für Netzwerke. Wir wollen im folgenden Konzepte, Technologien und Strategien beleuchten und erklären.



Konzepte der IT Security

1. CIA
2. Defense in depth
3. Secure by design
4. Least Privilege
5. Kerckhoffs' Prinzip
6. Zero Trust
7. KISS

CIA

CIA – Grundprinzipien der Informationssicherheit

- **Confidentiality:** Schutz vor unautorisiertem Zugriff (Least Privilege, Verschlüsselung, Isolation)
- **Integrity:** Sicherstellen, dass Daten unverändert und korrekt bleiben (Hashing, Signaturen, Prüfsummen)
- **Availability:** Systeme und Daten sind erreichbar und funktional (Redundanz, Failover, DoS-Schutz)
- Relevanz über **sämtliche IT-Schichten** hinweg

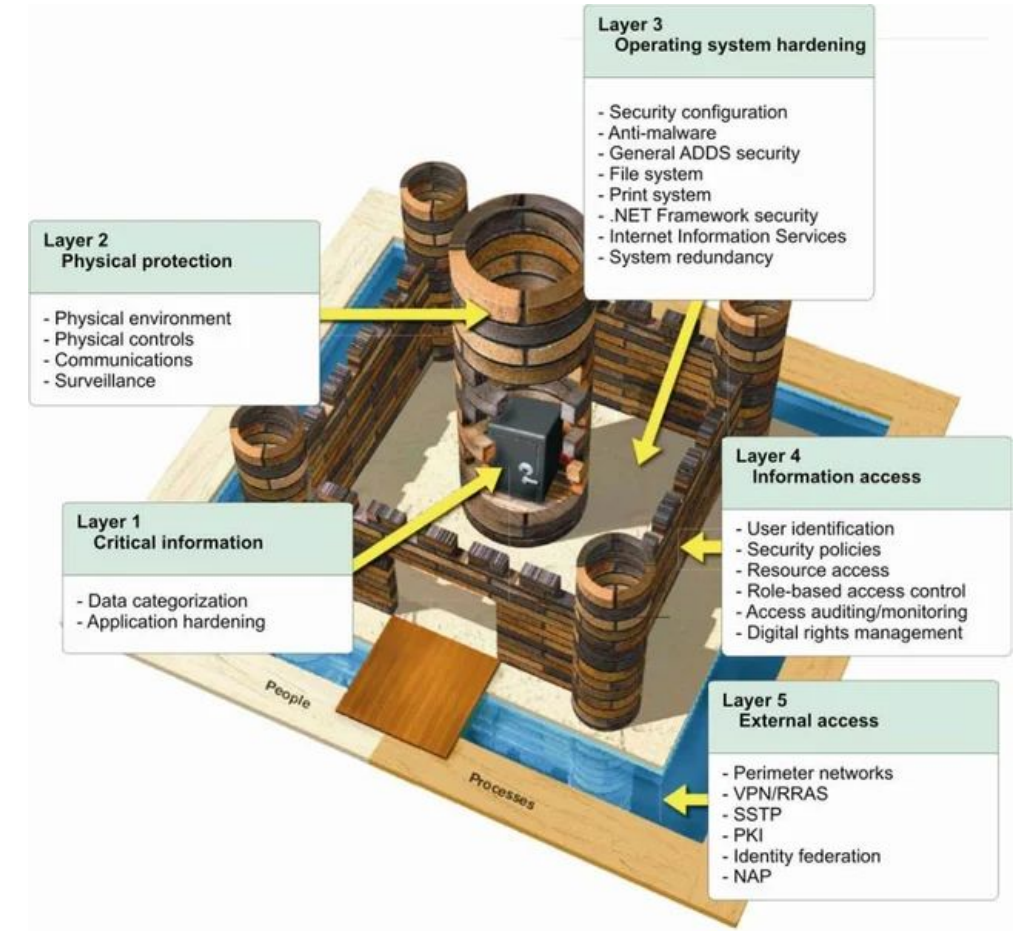
Beispiel: Wir bauen ein API-Gateway: Was müssen wir beachten?



Defense in Depth

Mehrschichtige Sicherheitsstrategien

- Kombination mehrerer unabhängiger Schutzmechanismen
- **Ziel: Kompensation einzelner Schwächen** durch mehrere Kontrollen
- Technische Ebenen: Netzwerk, Endpoint, Identität, Anwendung, Daten
- Organisatorische Ebenen: Prozesse, Monitoring, Schulungen
- Wichtig: Kontrolle möglichst heterogen, nicht identisch
- Reduziert die Wahrscheinlichkeit eines **Single Point of Failure**

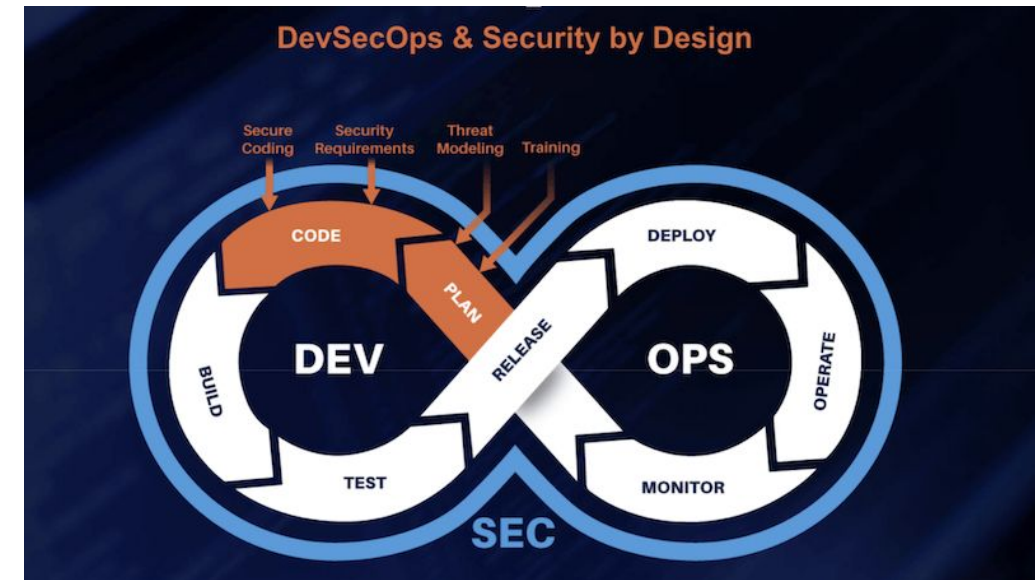


Beispiel: Wir sind IT Leiter im KMU. Was für Komponenten setzen wir für unseren Webserver ein?

Secure by Design

Sicherheitsprinzipien in Architektur & Entwicklung

- Sicherheit als integraler Bestandteil des Designs — kein Add-on
- Minimierung der Angriffsfläche (kleine Komponenten, wenig Exposed Services)
- Threat Modeling vor der Implementierung
- Standardisierte, gehärtete Basiskonfigurationen
- Sichere Defaults (deny-first, least functionality)
- Automatisiertes Testing (Static/ Dynamic Code Analysis)
- Vermeidung komplexer, schwer auditierbarer Systeme

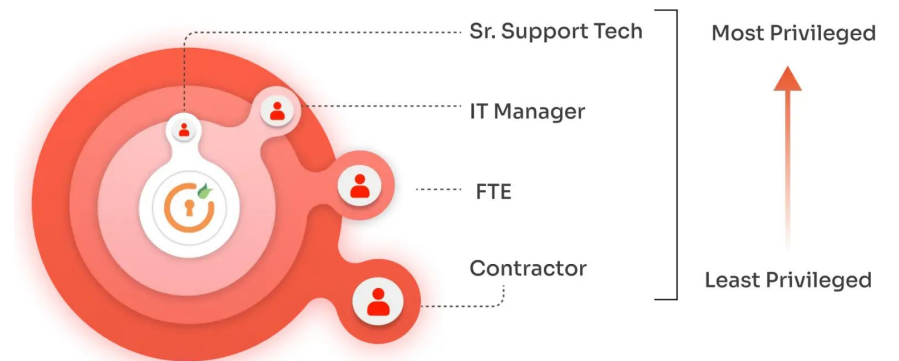


Beispiel: Wir entwickeln eine Buchhaltungssoftware. Auf was achten wir?

Least Privilege

Minimale Berechtigungen für Benutzer, Dienste & Systeme

- Jeder Akteur erhält nur exakt die Rechte, die notwendig sind
- Reduktion von lateral movement & Schadensradius
- Rollenbasiertes Berechtigungsmodell (RBAC)
- Regelmässige Reviews & Rezertifizierungen
- Prinzip gilt auch für Maschinenzugriffe, APIs, Secrets & Service-Accounts
- Technische Umsetzung: PAM, JIT Access, Segmentierung, ACLs



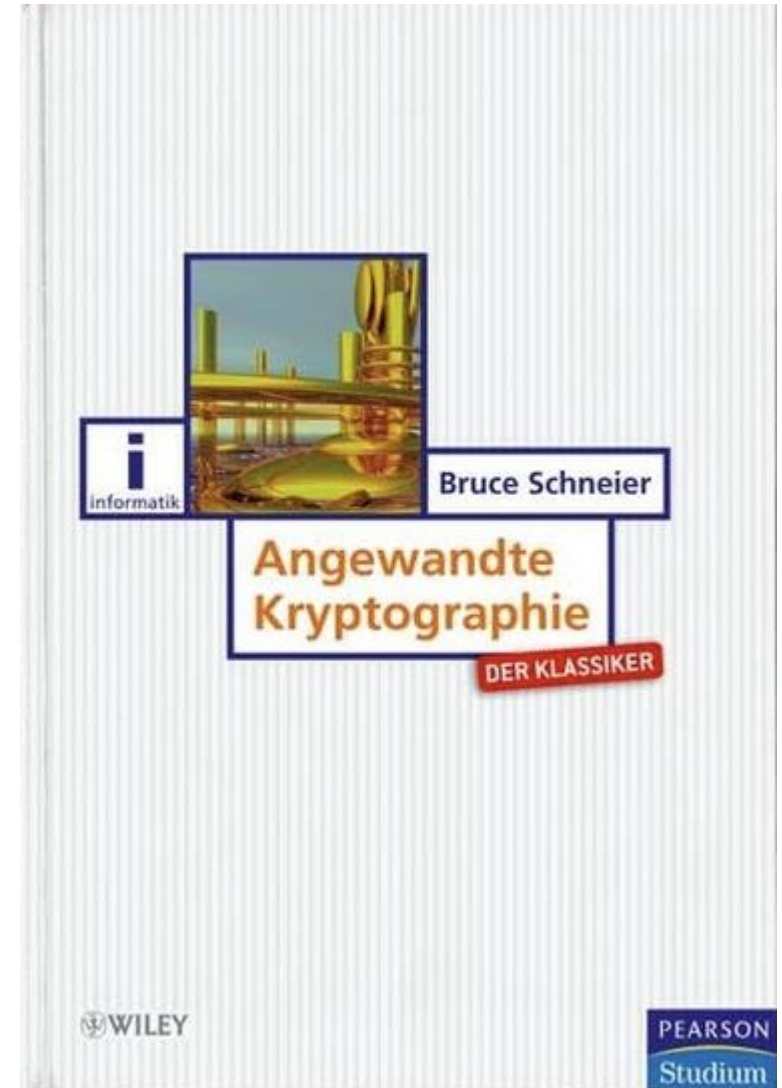
Beispiel: Wir Benutzen verwenden eine SSE Komponente für den Remote Zugriff auf interne Ressourcen. Worauf achten wir?

Kerckhoffs Prinzip

Sicherheitsprinzip aus der Kryptologie

- Sicherheit basiert auf dem Schlüssel, nicht auf Geheimhaltung des Verfahrens
- Angenommen: Angreifer kennt ganze Architektur & Implementierung
- Folge: Systeme müssen öffentlich überprüfbar & auditierbar sein
- Wichtig bei: Kryptografie, Authentifizierungsmechanismen, Protokollen
- Proprietäre „Security by Obscurity“-Mechanismen gelten als unsicher
- Fördert Transparenz, Robustheit, Peer Review

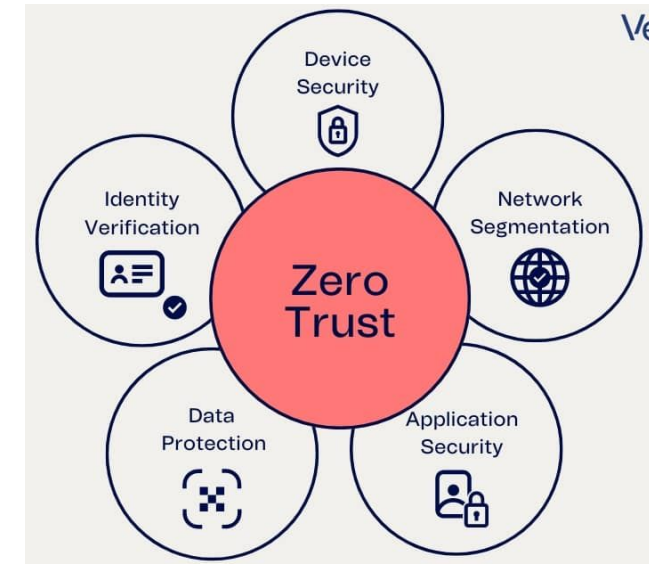
Beispiele für Nichteinhaltung des Prinzips?



Zero Trust

Identitätszentriertes Sicherheitsmodell

- Grundannahme: „Trust no one, verify everything“
- Keine impliziten Vertrauenszonen (LAN ≠ „trusted“)
- Zugriff basiert auf Identität, Gerätestatus, Kontext & Risiko
- Mikrosegmentierung von Anwendungen & Workloads
- Kontinuierliche Überprüfung statt einmaliger Authentifizierung
- Minimiert Angriffsfläche & lateral movement
- Technische Basis: ZTNA, IdP, Device Posture, Policies



Welche Probleme bringt Zero Trust (Network Access) mit sich?

KISS - Keep It Simple, Stupid

Reduktion von Komplexität in Sicherheitsarchitekturen

- Komplexe Systeme sind schwer prüfbar, schlecht betreibbar & anfälliger für menschliches Versagen
- Klare, minimalistische Designs reduzieren Fehlkonfigurationen
- Wenige, standardisierte Technologien statt heterogener Landschaft
- Kürzere Fehlersuche, bessere Auditierbarkeit
- Besonders relevant für Netzwerk- & Security-Architektur
- Ziel: hohe Stabilität bei minimaler Komplexität

THE KISS PRINCIPLE

**KEEP
IT
SIMPLE,
STUPID**

Was machen Benutzer mit komplizierten Systemen und Prozessen?

Weitere Prinzipien

- Fail-Safe Defaults (Default Deny)
 - Standardzustand ist verweigern, nicht erlauben
- Separation of Duties
 - Kritische Funktionen erfordern mehrere unabhängige Rollen
- Psychological Acceptability
 - Sicherheitsmechanismen müssen für Nutzer praktikabel sein
- Minimal Exposure
 - Nur unbedingt notwendige Dienste exponieren
- Time-based Security
 - Fokus auf „Detection + Response“ statt 100% Prävention

Angriffsvektoren

Welche Informationen findet man über Unternehmen?

- Basisdaten & Struktur
 - Name, Adresse, Handelsregistereintrag, Umsatz-/Jahresberichte
 - Standorte, Tochterfirmen, Partner & Lieferanten
- Technische Informationen
 - Domains, Subdomains, IP-Adressen, DNS-Einträge (MX, SPF, DKIM, TXT)
 - Offene Ports & Dienste (Webserver, VPN, Mailserver, RDP, ...)
 - Verwendete Technologien (CMS, Frameworks, Cloud-Anbieter, Versionsnummern)
- Mitarbeiter & Organisation
 - Namen, Rollen & Abteilungen (LinkedIn, Xing, Firmenwebsite)
 - E-Mail-Formate (vorname.nachname@firma.ch)
 - Organigramme, Zuständigkeiten, Funktionsbeschriebe
- Prozesse & Arbeitsweise
 - Stellenausschreibungen (genutzte Tools, interne Systeme, Sicherheitsniveau)
 - Veröffentlichte Richtlinien, Zertifizierungen (ISO 27001, etc.)
 - Öffnungszeiten, On-Site-Regeln, Besucherprozesse
- Dokumente & Metadaten
 - PDFs, Präsentationen, Whitepaper, Protokolle
 - Metadaten: Usernamen, interne Pfade, Softwareversionen, Erstellungszeitpunkte

Welche Angriffsvektoren ergeben sich daraus?

- Gezieltes Social Engineering
 - Überzeugende Phishing- und Betrugsszenarien anhand von Rollen, Projekten, Sprache
- Technische Angriffe auf Infrastruktur
 - Ausnutzen bekannter Schwachstellen in identifizierten Systemen und Diensten
- Missbrauch von Identitäten & Vertrauen
 - Täuschung im Namen von Führungskräften, Mitarbeitenden oder Geschäftspartnern
- Angriffe über Lieferkette & Integrationen
 - Kompromittierung schwächer geschützter Dienstleister und angebundener Systeme
- Ausnutzung organisatorischer Schwächen
 - Lücken in Prozessen, Freigaben, physischen Zugängen und Support-Strukturen
- Strategische Informationsnutzung
 - Aufbau von Profilen, langfristige Spionage, Erpressung und Reputationsangriffe

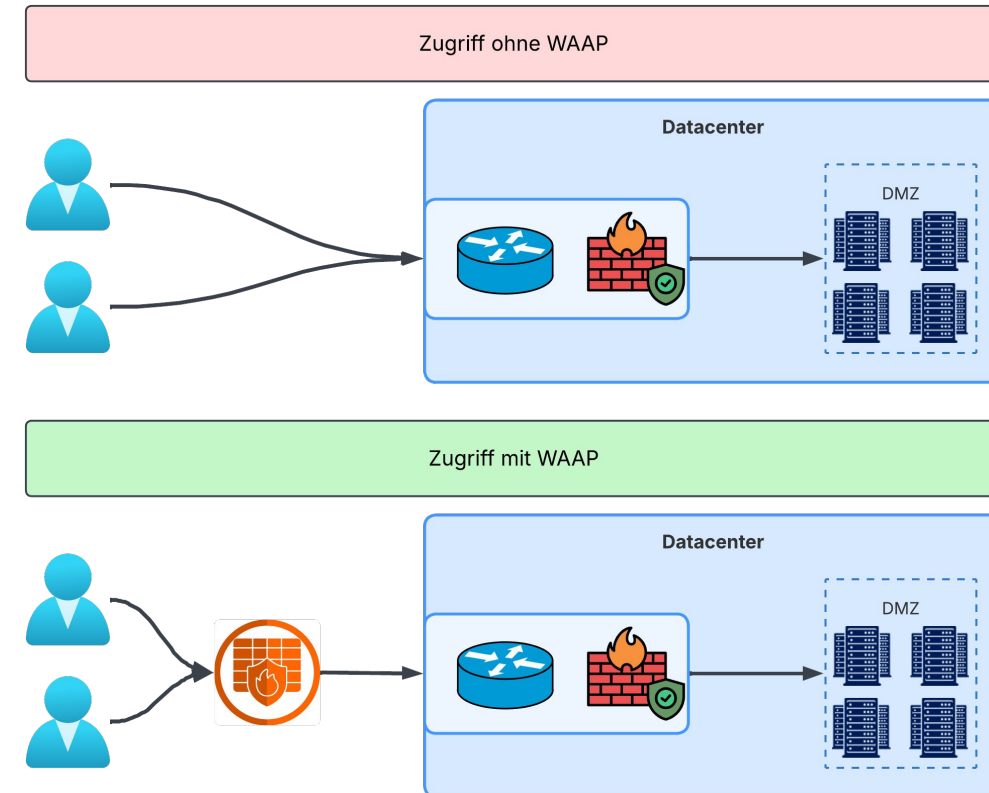
Fokus auf technische Angriffsvektoren

- Webanwendungen & APIs
 - Identifizierte Domains, Subdomains, Frameworks und Versionen → Ausnutzen bekannter Schwachstellen (z.B. SQLi, XSS, RCE)
- Exponierte Netzwerkdienste
 - Offene Ports und Services (VPN, RDP, SSH, Datenbanken) → Brute-Force, Exploits, Standard- oder Default-Logins
- Authentifizierung & Accounts
 - Gefundene Login-Portale und E-Mail-Schemata → Credential Stuffing, Passwort-Spraying, Angriffe auf schwache oder wiederverwendete Passwörter
- E-Mail- & Domain-Infrastruktur
 - Fehlkonfigurierte SPF/DKIM/DMARC-Records → Spoofing, technische Vorbereitung für Phishing und BEC
- Cloud- und SaaS-Umgebungen
 - Öffentlich zugängliche Buckets, fehlerhafte Freigaben, offene Admin-Panels → Datenabfluss und Rechteauserweiterung
- Automatisierte Reconnaissance
 - Nutzung von Scannern und Suchmaschinen (Shodan, Censys, etc.) → Systematisches Aufbauen eines „Target-Katalogs“ für spätere Angriffe

Web Application and API Protection (WAAP)

Was ist WAAP System?

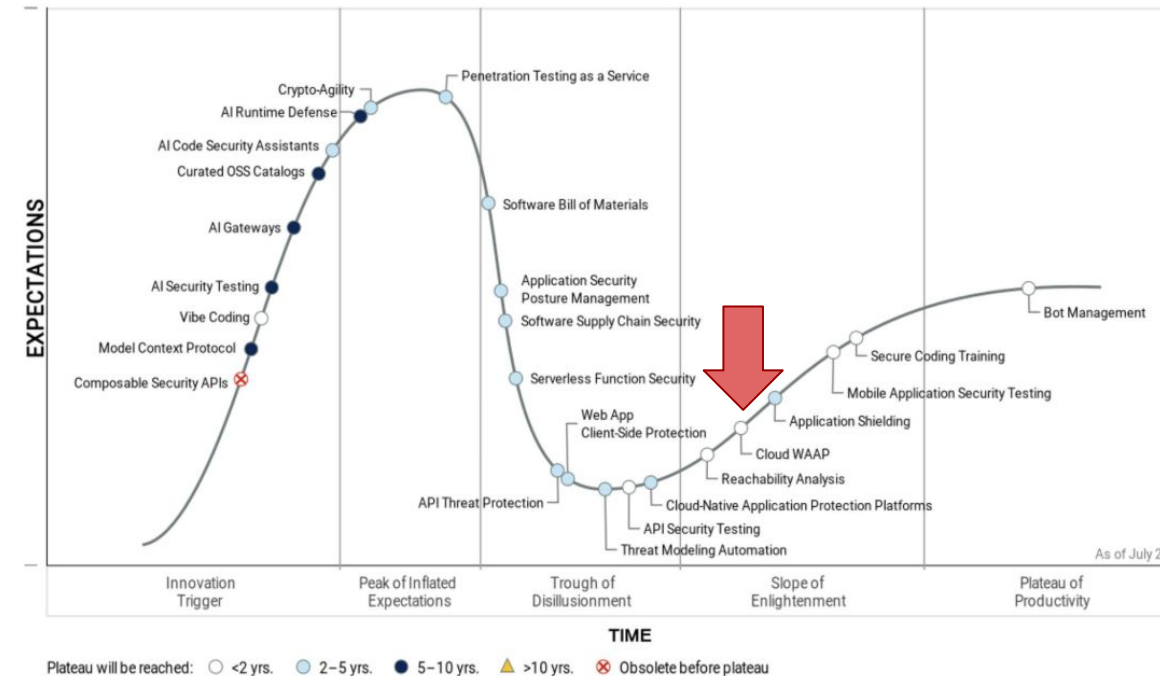
- Spezialisiertes Sicherheitssystem für HTTP- und API-Traffic
- Arbeitet auf Layer 7 des OSI-Modells
- Analysiert Anfragen und Antworten auf inhaltlicher Ebene
- Erkennungslogik: Signaturen, Heuristik, Parsing von Protokollfeldern
- Einsatz seit späten 1990ern, ursprünglich für Webserver-Härtung
- Primärer Zweck: Schutz vor anwendungsbezogenen Angriffen (Input-Manipulation, Exploits)



Unterschied: Firewall vs. WAAP

- Firewall:
 - Filtert nach IP, Port, Protokoll
 - Kein Verständnis von Applikationslogik
 - Bricht so gut wie nie TLS auf
- WAAP:
 - Interpretiert HTTP- und API-Requests & -Responses
 - Kennt Parameter, Header, Payload, Cookies, Sessions
 - Fokus auf Applikationslogik statt Netzwerkverkehr

Hype Cycle for Application Security, 2025



Typische Angriffsklassen

- Injection-Angriffe (SQLi, Command Injection, API Injection)
- Cross-Site Scripting (Xss)
- File Inclusion / Path Traversal
- Protocol & Spec Violations
- HTTP Desync / Request Smuggling
- Brute-Force und Credential-Stuffing (rate-based)
- Anormale Payloads (Encoding-Tricks, Obfuscation)
- Anfragen, die bekannte Schwachstellen triggern (CVEs)

Stärken und Grenzen

Stärken

- Schutz vor OWASP-Top-10-Angriffen
- Schutz auch bei ungepatchten Applikationen
- L7-Kontext statt einfacher Paketfilterung
- Zentrale Kontrollinstanz vor kritischen Webdiensten
- Kann API-Secrets zentral verwalten

Grenzen

- Keine vollständige Kompensation unsicherer Applikationen
- Umgehbar durch obskure Encodings oder Desync-Techniken
- False Positives bei dynamischen Web-Apps
- Regelpflege & Tuning erforderlich (hoher Admin-Aufwand)
- Hilft meistens nicht gegen Zero Day Exploits

Moderne WAAPs im Enterprise-Umfeld

- Übernehmen meist auch Load Balancing Funktionen
- Dienen häufig auch als CDN
- Übernehmen DDoS Funktionen (gerade Cloud-WAFs)
- Authentifizieren Benutzer & Maschinen
- Schützen APIs
- Schützen vor Bots
- Haben Sandboxes um Datei-Uploads zu inspizieren

Wie können Content-Creators vor dem Datenmissbrauch durch AI geschützt werden?

Filterung von AI Crawler

- AI Crawler halten sich nicht an die robots.txt (Datei die Crawlern sagt, was sie verwenden sollen und was nicht)
- Problem: Content-Creator die wertvolle Beiträge und Wissen erarbeiten, werden nicht mehr vorgestellt und erhalten keine Seitenaufrufe mehr, wenn die AI dieses Wissen einfach in seinen Antworten verwendet.
- Überlegung: Man identifiziert AI-Crawler auf der WAAP und erlaubt dem Dateneigentümer (Websitebetreiber) selbst zu entscheiden, was die AI harvesten darf und was nicht.
- 3 Mögliche Optionen:
 - Allow: AI darf den Content kostenlos verwenden
 - Charge: AI muss bezahlen, damit sie den Content verwenden darf
 - Block: AI darf den Content nicht einsehen

Filterung von AI Crawler

AI Crawl Control

Analyze and block known AI crawlers from scraping your website.

[AI Crawl Control documentation](#)

Crawlers Metrics Robots.txt Settings

Nov 23, 12PM (UTC) - Nov 30, 12PM (UTC)

Requests

371
Total
319
Allowed
52
Unsuccessful

Crawlers

26
Total
25
Allowed
1
Blocked

Robots.txt

 Self Managed 

☐	Crawler	Category	Requests	Action
☐	BingBot Microsoft	Search Engine Crawler	 Allowed: 86 Unsuccessful: 1	<button>Allow</button> <button>Block</button> ⋮
☐	GPTBot OpenAI	AI Crawler	 Allowed: 47 Unsuccessful: 34	<button>Allow</button> <button>Block</button> ⋮
☐	Googlebot Google	Search Engine Crawler	 Allowed: 63 Unsuccessful: 7	<button>Allow</button> <button>Block</button> ⋮
☐	Claude-User Anthropic	AI Crawler	 Allowed: 55 Unsuccessful: 2	<button>Allow</button> <button>Block</button> ⋮
☐	ChatGPT-User OpenAI	AI Assistant	 Allowed: 26 Unsuccessful: 0	<button>Allow</button> <button>Block</button> ⋮
☐	Bytespider ByteDance	AI Crawler	 Allowed: 17 Unsuccessful: 7	<button>Allow</button> <button>Block</button> ⋮
☐	Meta-ExternalAgent Meta	AI Crawler	 Allowed: 10 Unsuccessful: 1	<button>Allow</button> <button>Block</button> ⋮
☐	ClaudeBot Anthropic	AI Crawler	 Allowed: 6 Unsuccessful: 0	<button>Allow</button> <button>Block</button> ⋮
☐	PerplexityBot Perplexity	AI Search	 Allowed: 4 Unsuccessful: 0	<button>Allow</button> <button>Block</button> ⋮

AI Crawl Control

Analyze and block known AI crawlers from scraping your website.

[AI Crawl Control documentation](#)

Crawlers Metrics Robots.txt **Settings**

Block response

Configure the response that blocked crawlers will receive when they attempt to access your website.

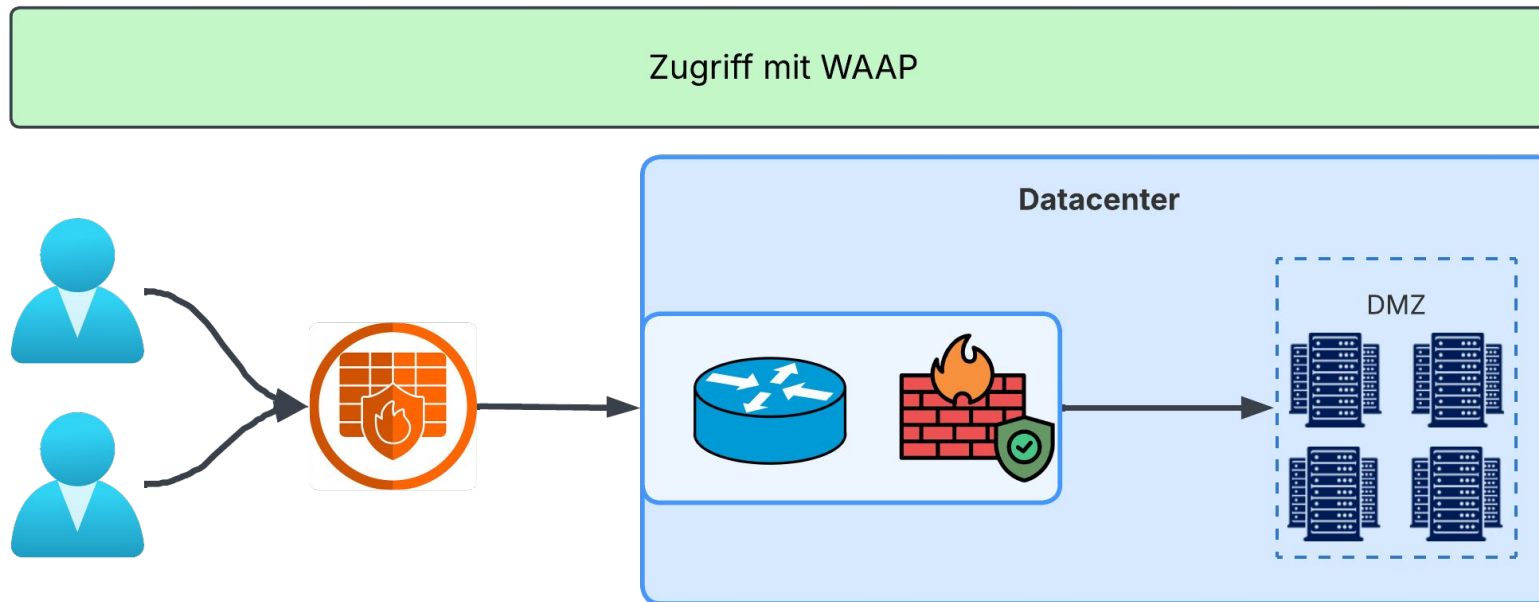
Response code

402 Payment Required

402 Payment Required

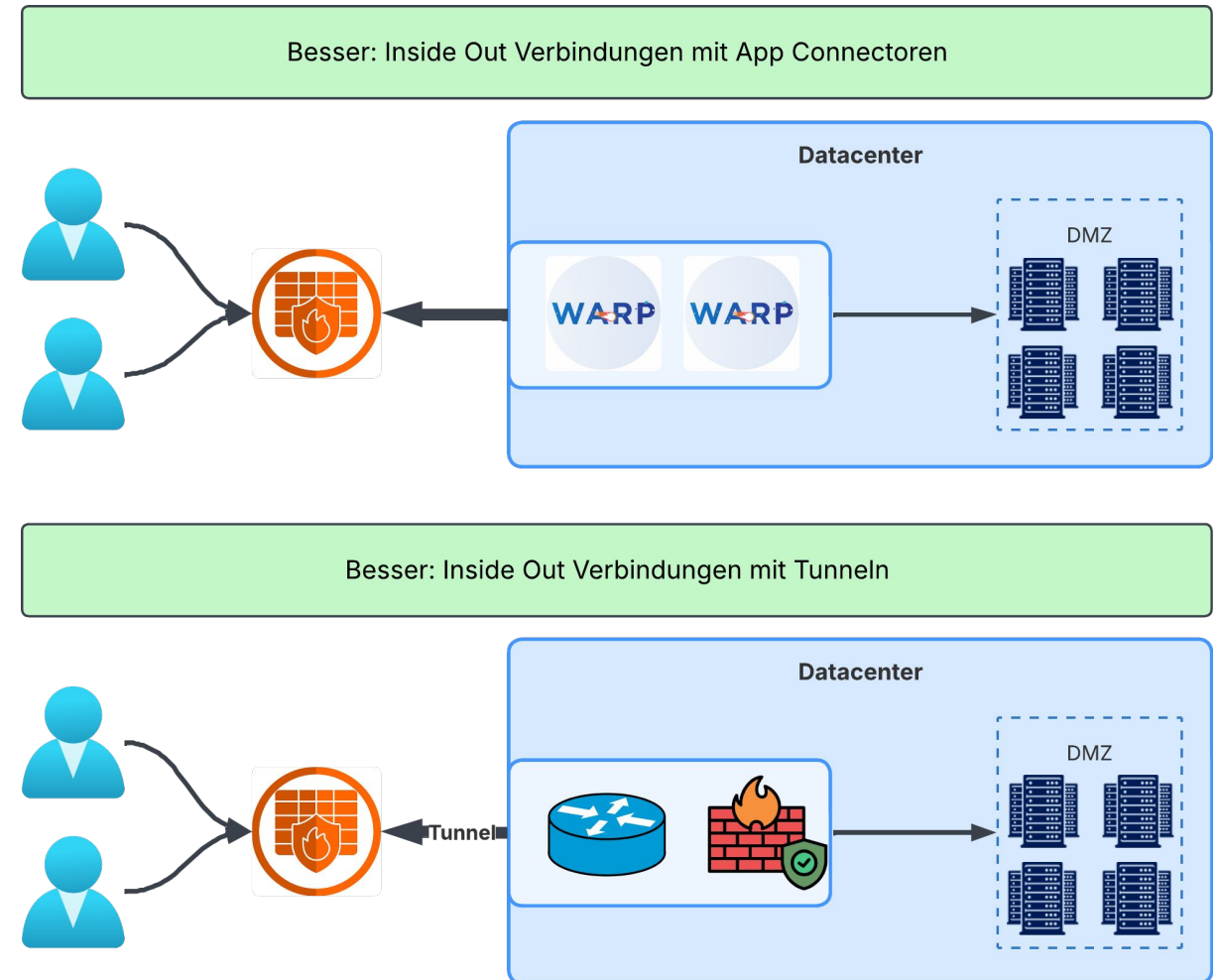
403 Forbidden

Frage: Wie greifen wir dieses System an?



Besser: Inside Out Verbindungen

- Anstatt Ports für die DMZ von aussen (für die WAAP) zugänglich zu machen, werden die Verbindungen zur WAAP von innen initiiert:
 - Über Tunnel (GRE / VPN) die der Perimeter aufbaut, worüber die WAAP die DMZ erreichen kann
 - Über App Connectoren, die in der DMZ aufgebaut werden und Verbindungen zur WAAP aufbauen und ihr gleichzeitig erlauben auf DMZ Dienste zu zugreifen



E-Mail Sicherheit

Bedeutung von E-Mail-Sicherheit

Warum E-Mail weiterhin ein kritischer Angriffsvektor ist

- Über 80 % aller Cyberangriffe beginnen per E-Mail
- Häufige Angriffsformen: Phishing, Malware, Social Engineering, Account-Takeover
- Keine Protokollauthentizität im SMTP-Design (1970er-Standard)
- Hohe Anzahl an Drittanbietern, Weiterleitungen & Mail-Relays erhöht Komplexität
- Unternehmen benötigen Schutzmechanismen auf Transport-, Inhalts- und Identitätsebene

Problem: E-Mail ist ein Uralt Protokoll, bei dessen Erfindung niemand an Sicherheit gedacht hat.

Wichtige Schutzmechanismen (Überblick)

Technische Massnahmen

- SPF, DKIM, DMARC zur Authentifizierung & Policy Enforcement
- Malware- und Spamfilter (Heuristik, Signaturen, ML)
- Sandboxing für Anhänge & verdächtige Links
- TLS-Erzwingung für Transportverschlüsselung
- DANE/DNSSEC für zusätzliche Integrität des Mail-Routings
- SMIME / PGP für Email-Verschlüsselung

Organisatorische Massnahmen

- Awareness-Trainings (regelmässig, realitätsnah, adaptiv)
- Klare und einfache Prozesse für Meldung und Eskalation verdächtiger Nachrichten

Entitäten im Mail Ökosystem

MTA - Mail Transfer Agent

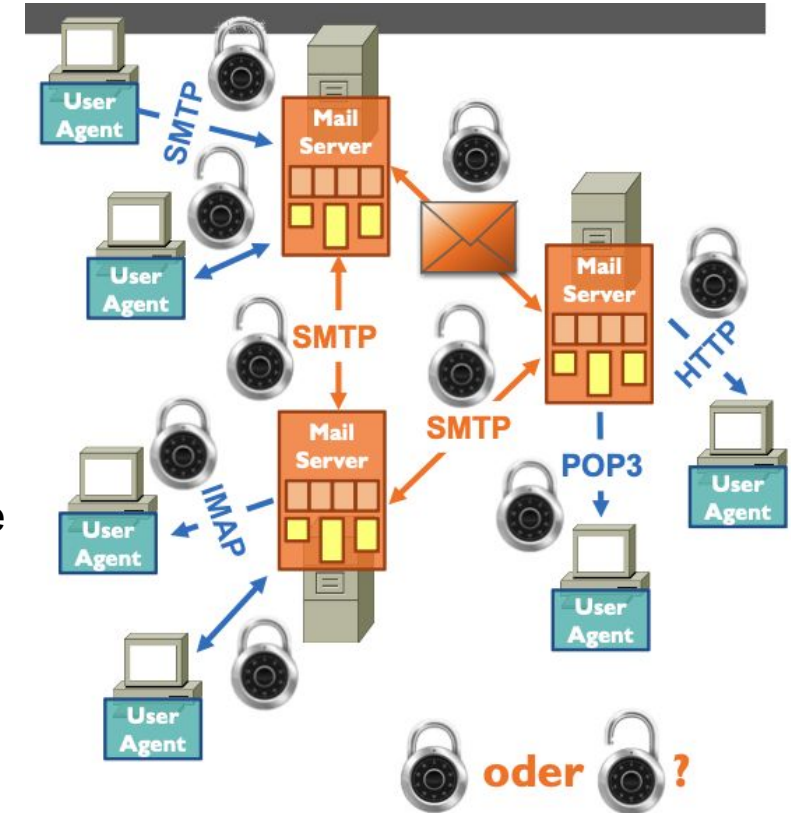
- Nimmt E-Mails entgegen und transportiert sie zwischen Servern (Domain → Domain) (Postfix, Exim, Exchange Online, etc.)
- Protokolle: SMTP(S) (Port 25), STARTTLS (Addon-Protokoll für Port 25)

MDA - Mail Delivery Agent

- Übergibt die E-Mail ins Postfach des Empfängers (Dovecot, Procmail, Exchange Mailbox Transport, etc)
- Protokolle: LMTP, LDA. Der Ort, wo Spam, Virus und DKIM geprüft wird bevor die Mail zugestellt wird

MUA - Mail User Agent

- Email-Client: Schreiben, Lesen, Senden, Empfangen durch den Nutzer (Apple Mail, Outlook, Gmail-App, etc)
- Protokolle: IMAP, POP3



Port Übersicht

Port	Name	Zweck	Authentifizierung	TLS
25	SMTP (MTA)	Server-zu-Server	SPF, DKIM, DMARC, DANE	optional (STARTTLS)
587	SMTP Submission	Client-zu-Server	SMTP AUTH (PLAIN, SCRAM, OAuth2)	STARTTLS Pflicht
465	SMTPS	Client-zu-Server	SMTP AUTH (PLAIN, SCRAM, OAuth2)	Implicit TLS

SPF, DKIM, DMARC, DANE (Identitätsprüfung)

SPF (Sender Policy Framework)

- Definiert autorisierte Mail-Server einer Domain
- Schutz gegen Absender-Spoofing
- Limitierung: Keine Prüfung der Display-From-Adresse (nur Envelope-From)

DKIM (DomainKeys Identified Mail)

- Digitale Signatur im Header
- Integrität & Authentizität der Nachricht
- Schutz vor Manipulation auf Transportwegen

Welche Attribute des CIA-Paradigmas werden damit geschützt?

SPF, DKIM, DMARC, DANE (Identitätsprüfung)

DMARC (Domain-based Message Authentication, Reporting & Conformance)

- Verknüpft SPF & DKIM mit Policy (none/quarantine/reject)
- Schutz vor direktem Domainmissbrauch
- Ermöglicht Reporting und Auswertung von Missbrauchsmustern

DANE (DNS-Based Authentication of Named Entities)

- Bindet das TLS-Zertifikat einer Domain indem es einen zusätzlichen DNS-Record (mit DNSSEC!) publiziert, der sagt, welches Zertifikat ein Empfänger-Server für die Verschlüsselung zu erwarten hat.
- Verhindert Man-in-the-Middle durch gefälschte TLS-Zertifikate
- Stellt sicher, dass TLS beim SMTP-Transport authentisch ist

Wir bauen Domain-Authentication mit SPF, DKIM & DMARC

Wer hat eine eigene mail-domain?

Verschlüsselung

- Bietet Vertraulichkeit durch das Verschlüsseln der Nachrichten.

Merkmale	S/MIME	OpenPGP
Architektur	PKI (hierarchisch)	Web of Trust (dezentral)
Verbreitung	Unternehmen	technisch versierte Nutzer
Schlüsselverteilung	Automatisiert über zentrale Infrastruktur	Manuell oder über öffentliche Keyserver
RFC	RFC 2633	RFC 2440

Verschlüsselung

Merkmale	S/MIME	OpenPGP
Architektur	PKI (hierarchisch)	Web of Trust (dezentral)
Verbreitung	Unternehmen	technisch versierte Nutzer
Schlüsselverwaltung	Zentral über CAs	Manuell oder über öffentliche Keyserver
RFC	RFC 2633	RFC 2440

- Bietet Vertraulichkeit durch das Verschlüsseln der Nachrichten.
- Drei Ansätze: S/MIME, PGP und TLS
- S/MIME und PGP erlauben ende-zu-ende Verschlüsselung. TLS nur zwischen MTA & MTA
- In der Praxis:
 - Ist PGP für die meisten zu komplex. Daher ist die Verbreitung und damit Unterstützung sehr gering
 - Ist S/MIME meist im stark regulierten Unternehmen (Banken, Versicherungen, Gesundheit) im Einsatz, aber häufig beschränkt auf Domain2Domain (mit einer zentralen instanz, die das Verschlüsseln von Mails auf Domain-Ebene erzwingt, aber nicht auf User-Ebene)
 - Ist TLS die gängigste Art, Mails zwischen MTAs zu verschlüsseln. Für die meisten Unternehmen reicht das.

Frage: Wie sieht eine Mail-Infrastruktur aus, die mit S/MIME Domain-Verschlüsselung arbeitet?



Open Relay

Ein Open Relay ist ein Mailserver, der E-Mails von jedem Absender an jeden Empfänger weiterleitet, ohne Authentifizierung oder Einschränkungen.

Auswirkung:

- Wird sofort von Spammern missbraucht
- Server landet auf Blacklists → legitime Mails werden blockiert
- Hohe Last, IP-Blockierung, Reputationsschäden
- Sicherheitsrisiko für das gesamte Netzwerk

Richtige Konfiguration für MTAs:

- Authentifizierte SMTP-Submission (Port 587/465)
- Port 25: Kein SMTP-Auth, Prüfung von DMARC, SPF & DKIM und KEIN RELAYING zu anderen Domains
- SPF, DKIM, DMARC + Logging für eigene Domain
- Regelmässige Relay-Tests / Monitoring

Open Relay Test



- Lookup des mailservers:
 - dig MX rheintec.ch +short -> 10 de-smtp-inbound-1.mimecast.com.
- Verbindung auf Port 25:
 - telnet de-smtp-inbound-1.mimecast.com 25
- Anmeldung des Senders:
 - HELO test.attack.com
- Benennung des Senders:
 - MAIL FROM:<linus.espach@rheintec.ch>
 - RCPT TO:<finance@customer.ch>

```
[linus@nba-MW3304CK95 ~ % dig MX rheintec.ch +short
10 de-smtp-inbound-1.mimecast.com.
10 de-smtp-inbound-2.mimecast.com.
linus@nba-MW3304CK95 ~ % telnet de-smtp-inbound-1.mimecast.com 25
Trying 194.104.110.22...
Connected to de-smtp-inbound-1.mimecast.com.
Escape character is '^]'.
220 de-smtp-inbound-1.mimecast.com ESMTP; Sun, 30 Nov 2025 15:44:35 +0100
HELO test.attack.com
250 de-smtp-inbound-1.mimecast.com Hello [85.195.247.251]
MAIL FROM:<linus.espach@rheintec.ch>
250 Sender OK [a97trvM_NaWZpDBmVBbhUQ.de86]
RCPT TO:<finance@customer.ch>
451 Open relay not allowed - https://community.mimecast.com/docs/DOC-1369#451 [a97trvM_NaWZpDBmVBbhUQ.de86]
```

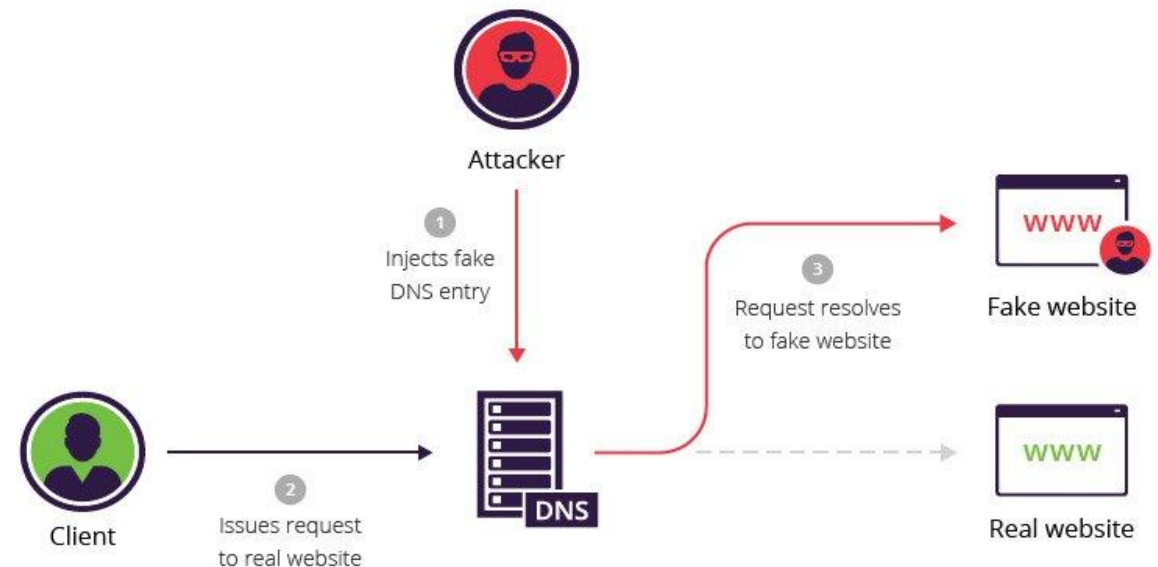
DNS Security & DNSSEC

Angriffe: DNS-Spoofing / DNS-Cache-Poisoning / Man-in-the-Middle

1. Angreifer schickt gefälschte DNS-Antworten, damit dein Resolver falsche IPs speichert.
2. Nutzer ruft bank.ch auf
3. Gefälschte DNS-Antwort → falsche IP
4. Nutzer landet auf Phishing-Seite

Aber:

- Dies fälscht nur die DNS-Antwort
- Ein Angreifer braucht auch ein Zertifikat



Lösung: DNSSEC

- Schützt das DNS vor falschen Antworten mittels kryptographischer Signatur (Integrität & Authentizität)
- Teilnehmer:
 - **DNSKEY**: Öffentlicher Schlüssel einer Zone (z.B. [example.com](https://www.example.com))
 - **RRSIG**: Die Signatur die an jede DNS-Antwort angehängt wird (signiert vom DNS-Server mit dem Private Teil des DNSKEYs)
 - **DS-Entry**: Der Hash des des DNSKEY in der übergeordneten Zone (hier: .com) für die Chaine-of-Trust

Wichtig:

- Ein DNS-Resolver muss “Validierend” sein, damit DNSSEC funktioniert, weil ein Angreifer ansonsten einfach die Signaturen “Strippen” könnte, da Clients aufgrund der rekursiven Funktion von DNS ansonsten nichts von der Existenz von DNSSEC mitbekommen.
- Daher ist es wichtig, validierende Resolver auf Perimetern einzusetzen, z.B.: 1.1.1.1 & 8.8.8.8

Tool: DNSVIZ & Verisign Labs



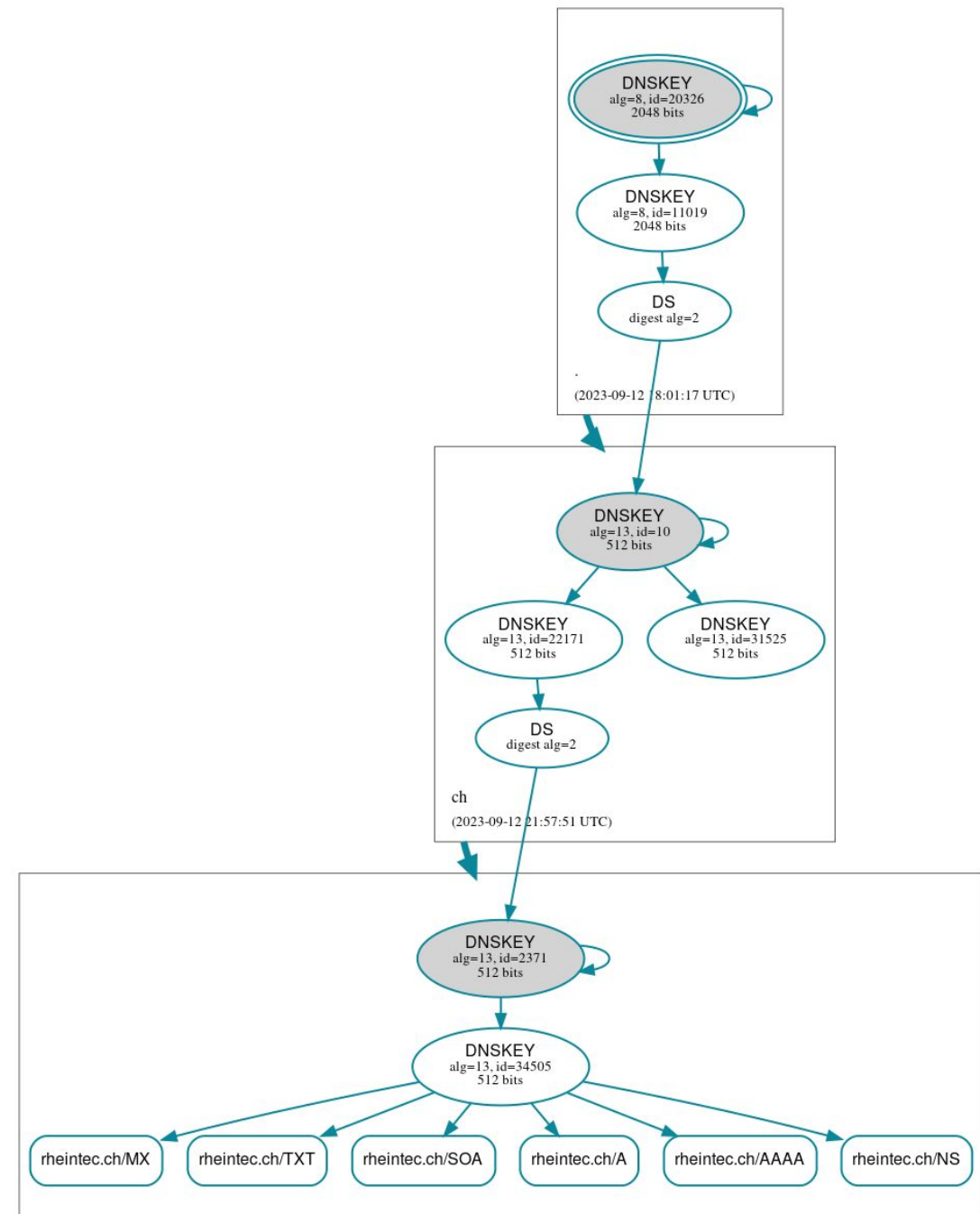
Domain Name:

Analyzing DNSSEC problems for [rheintec.ch](#)

.	- Found 3 DNSKEY records for . - DS=20326/SHA-256 verifies DNSKEY=20326/SEP - Found 1 RRSIGs over DNSKEY RRset - RRSIG=20326 and DNSKEY=20326/SEP verifies the DNSKEY RRset
ch	- Found 1 DS records for ch in the . zone - DS=58745/SHA-256 has algorithm ECDSAP256SHA256 - Found 1 RRSIGs over DS RRset - RRSIG=61809 and DNSKEY=61809 verifies the DS RRset - Found 4 DNSKEY records for ch - DS=58745/SHA-256 verifies DNSKEY=58745/SEP - Found 2 RRSIGs over DNSKEY RRset - RRSIG=14051 and DNSKEY=14051/SEP verifies the DNSKEY RRset
rheintec.ch	- Found 1 DS records for rheintec.ch in the ch zone - DS=2371/SHA-256 has algorithm ECDSAP256SHA256 - Found 1 RRSIGs over DS RRset - RRSIG=44426 and DNSKEY=44426 verifies the DS RRset - Found 2 DNSKEY records for rheintec.ch - DS=2371/SHA-256 verifies DNSKEY=2371/SEP - Found 1 RRSIGs over DNSKEY RRset - RRSIG=2371 and DNSKEY=2371/SEP verifies the DNSKEY RRset - nelly.ns.cloudflare.com is authoritative for rheintec.ch - rheintec.ch A RR has value 199.60.103.182 - Found 1 RRSIGs over A RRset - RRSIG=34505 and DNSKEY=34505 verifies the A RRset
rheintec.ch	- milan.ns.cloudflare.com is authoritative for rheintec.ch - rheintec.ch A RR has value 199.60.103.82 - Found 1 RRSIGs over A RRset - RRSIG=34505 and DNSKEY=34505 verifies the A RRset

Move your mouse over any 🐛 or 🛠 symbols for remediation hints.

Want a second opinion? Test rheintec.ch at [dnsviz.net](#).



Wieso ist DNSSEC gerade für Mailing wichtig?

- **Schutz der MX-Records**
 - Verhindert Umlenken von E-Mails auf gefälschte Server
 - Unterbindet Mail-Hijacking und stille MITM-Angriffe
- **Voraussetzung für DANE (TLSA)**
 - DNSSEC + DANE sichern SMTP-TLS kryptografisch ab
 - Verhindert STARTTLS-Downgrades und Zertifikats-Manipulation
- **Absicherung von SPF, DKIM & DMARC**
 - Schützt DNS-basierte Anti-Spoofing-Einträge vor Manipulation
 - Erhöht die Authentizität des E-Mail-Absenders
- **DNS = kritischer Bestandteil jeder Mailzustellung**
 - Bei jedem Schritt werden DNS-Einträge benötigt
 - Manipulation in DNS → komplette Kompromittierung des Mailflows

DNS Zonentransfers

- **Zonentransfer:**
 - Übertragung des Inhalts einer DNS-Zone von Primary- zu Secondary-Nameservern
- **Zweck:**
 - Redundanz, Lastverteilung
- **Transfertypen:**
 - AXFR (Full Zone Transfer)
 - IXFR (Incremental Zone Transfer)
- **Angriff:**
 - Angreifer versucht die Zone zu kopieren, um Informationen zu allen Einträgen zu erhalten
 - Wird verhindert, indem sich Clients beim Zonentransfer Authentifizieren müssen (IP oder HMAC-Key)

Test DNS Zonentransfers

- **Nameserver herausfinden:**
 - dig +short rheintec.ch NS
- **Zonentransfer anfragen:**
 - dig AXFR rheintec.ch @milan.ns.cloudflare.com

```
linus@nba-MW3304CK95 ~ % dig +short rheintec.ch NS
milan.ns.cloudflare.com.
nelly.ns.cloudflare.com.
linus@nba-MW3304CK95 ~ % dig AXFR rheintec.ch @milan.ns.cloudflare.com

; <>> DiG 9.10.6 <>> AXFR rheintec.ch @milan.ns.cloudflare.com
;; global options: +cmd
; Transfer failed.
linus@nba-MW3304CK95 ~ %
```

DNS Tunneling

- **Technik zur Datenübertragung über DNS-Anfragen und DNS-Antworten**
- **Missbraucht DNS als Transportkanal**, z.B. für:
 - Command & Control (C2) von Malware
 - Datenexfiltration (Umgehung von Firewalls und DLPs)
- **Funktion:**
 - Daten werden in DNS-Anfragen eingebettet, typischerweise in Subdomains (tunnel.example.com)
 - Der DNS-Resolver leitet die Anfrage rekursiv weiter, bis sie den autoritativen Nameserver des Angreifers erreicht.
 - Der autoritative Server extrahiert die eingebetteten Daten und kann in seiner DNS-Antwort wiederum Daten zurückkodieren.
 - Ergebnis: Es entsteht ein bidirektionaler Kommunikationskanal, der vollständig über Port 53 läuft

DNS Security

- Einsatz von vertrauenswürdigen, zentralen Unternehmens-Resolvern
 - Durchsetzung dieser Konfiguration auf allen Clients
 - keine direkten DNS-Anfragen ins Internet erlauben
- Security-Funktionen dieser Resolver
 - Domain-/Record-Analyse: Abgleich mit Threat-Intelligence (Malware-/Phishing-Domänen)
 - DoH: DNS over HTTPS
 - Erkennung von DNS-Tunneling: Analyse von Anomalien (lange Labels, hohe Entropie, ungewöhnliche Query-Frequenzen)
 - DNSSEC-Validierung: Erzwingt Authentizität von DNS-Antworten
 - Detection unauthorisierter Resolver: Erkennung von DoH/DoT/externen Resolvern (z. B. Tor, öffentliche DNS-Server)
- In ssE-/SASE-Umgebungen (Secure Service Edge) übernimmt die Cloud-Security-Komponente häufig diese Funktionen für Clients – unabhängig vom Standort

Vulnerability Management

Vulnerability Management: Ziel & Grundidee

- Kontinuierlicher Umgang mit technischen Schwachstellen in Systemen, Applikationen und Cloud-Umgebungen
- Ziel: Risiken durch bekannte Schwachstellen so weit wie möglich reduzieren
- Kernfragen:
 - Wo habe ich Schwachstellen?
 - Wie kritisch sind sie für mich?
 - Wer behebt sie bis wann?
- Braucht sowohl Technik (Scanner, Tools) als auch Prozesse & Verantwortung in den Fach- / IT-Teams

Vulnerability Management: Prozess

- Inventar
 - Systeme / Anwendungen erfassen, Owner & Kritikalität hinterlegen
- Erkennung
 - Regelmässige Schwachstellenscans (Netzwerk, Agent, Cloud, Container)
- Bewertung
 - Technische Schwere, Exponiertheit, Business-Relevanz zusammen betrachten
- Behebung
 - Patches einspielen, Konfigurationen anpassen, notfalls Workarounds
- Kontrolle & Reporting
 - Nachscans, SLA-Überwachung, Kennzahlen für IT / Management

Bausteine eines VM-Programms

- Organisation
 - Rollen (VM-Verantwortliche, System-Owner), Richtlinien, Remediation-SLAs
- Technik
 - Zentrale VM-Plattform, Scanner, Schnittstellen zu Ticketing / Change-Prozess
- Datenbasis
 - Aktuelles Asset-Inventory, Zuordnung zu Services / Verantwortlichen
- Governance
 - Regelmässige Reports, Review von Ausnahmen („kann nicht gepatcht werden“)

Vulnerability Management – Stärken & Schwächen

- Stärken
 - Liefert Transparenz über Schwachstellen-Lage
 - Erlaubt Priorisierung & messbaren Fortschritt (KPIs)
 - Unterstützt Compliance & Audits
- Schwächen / Herausforderungen
 - Unvollständiges Inventar → blinde Flecken
 - Viele Findings → Gefahr von Ticket-Flut & „Vulnerability Fatigue“
 - Legacy-/Sondersysteme lassen sich oft nicht einfach patchen
 - Fokus eher Onprem: Wird in Clouds durch CNAPPs abgelöst

CNAPP (Cloud-Native Application Protection Platform)

CNAPP – Cloud Native Application Protection Platform

- Plattform, um Cloud-native Anwendungen abzusichern:
 - Container, Kubernetes, Serverless, PaaS, Cloud-VMs, IaC ...
- Ziel: Sicherheit über den ganzen Lebenszyklus von
 - Code & Pipeline
 - über Cloud-Konfiguration
 - bis zur Laufzeit
- Typische Probleme, die CNAPP adressiert:
 - Fehlkonfigurierte Cloud-Ressourcen (S3 offen im Internet, falsche Security Groups ...)
 - Unsichere Container-Images / Libraries
 - Zu viele Rechte (IAM-Rollen, Service Accounts)
 - Verdächtigtes Verhalten zur Laufzeit
- Merksatz: CNAPP = „Rundumschutz“ für (ausschliesslich!) Cloud-Workloads und -Konfigurationen.

Bausteine einer CNAPP Plattform

- CSPM (Cloud Security Posture Management)
 - Prüft Cloud-Konfigurationen gegen Best Practices / Policies
 - z.B.: offene Buckets, unsichere Security Groups, fehlende Verschlüsselung
- CWPP (Cloud Workload Protection Platform)
 - Schutz der Workloads (VMs, Container, Serverless)
 - Vulnerability-Scanning, Laufzeit-Schutz, Prozess-/Netzwerk-Überwachung
 - Kubernetes / Container Security
 - Checks für Cluster-Konfiguration (RBAC, Namespaces, Netzwerk-Policies)
 - Kontrolle von Images (Signaturen, Schwachstellen, „Image Provenance“)
 - IaC- & Pipeline-Scanning
 - Scannt Terraform, CloudFormation, Helm-Charts, CI/CD-Pipelines
 - findet Sicherheitsprobleme, bevor sie deployed werden
- CIEM (Cloud Infrastructure Entitlement Management)
 - Analysiert IAM-Rechte (User, Rollen, Service Accounts)
 - findet „overprivileged“ Accounts („kann viel mehr, als er sollte“)

CNAPP – Stärken und Schwächen

- **Stärken**

- Konfig-Sicht: Findet Fehlkonfigurationen in Cloud-Accounts (z.B. offene Buckets, zu breite Security Groups).
- Runtime-Schutz: Überwacht laufende Workloads (Container/VMs) auf verdächtige Prozesse, Verbindungen, Policy-Verstöße.
- Shift Left: Scannt IaC/Container-Images schon in Git/CI/CD und warnt vor dem Deploy.

- **Grenzen**

- Cloud-fokussiert: Schützt vor allem Public-Cloud-Workloads – On-Prem/OT brauchen andere Lösungen.
- Keine Voll-Forensik: Ersetzt kein tiefes Host-/Netz-Monitoring (EDR, NDR, SIEM bleiben wichtig).
- Viele Findings: Ohne klare Verantwortliche & Prozesse bleiben Warnungen liegen → „Finding-Fatigue“.
- CNAPP schützt primär Public-Cloud-Workloads – klassische On-Prem-Server/OT werden typischerweise durch andere Lösungen adressiert (EDR, VM, NDR).

Vergleich von CNAPP & XDR

Kriterium	CNAPP	XDR
Hauptziel	Cloud-Konfig & -Workloads absichern	Angriffe (auf Clients & Servern) erkennen & stoppen
Datenquellen	Cloud-APIs, K8s, Container, IaC/CI/CD	EDR, Mail, Identity, Proxy/DNS, NDR
Blickwinkel	Cloud-Posture + Runtime-Schutz	Angriffsketten über Hosts, User, Netzwerk & Cloud
Typische Use Cases	Offene Buckets, falsche SGs, CVEs in Images, zu breite IAM-Rechte	Phishing, Kontoübernahme, Malware, C2, Lateral Movement
Integration / Zusammenspiel	DevSecOps-Tools; Findings an SIEM/XDR/SOAR	Kernsystem im SOC; triggert SOAR & Response

(Unified) Endpoint Management

Endpoint Management – Ziel & Grundidee

- Zentrales Verwalten von Endgeräten (Clients & Server)
- Einheitliche Konfiguration, Applikationen, Patches und Softwarestände sicherstellen
- Ziel: stabile, sichere und automatisiert betriebene Systeme
- Grundlage für: klare Zuständigkeiten & reproduzierbare Builds

Endpoint Management – Kernaufgaben

- Inventar & Lifecycle
 - Erfassen, klassifizieren, Nachverfolgen von Geräten (von Bereitstellung bis Ausmusterung)
- Konfigurationsmanagement
 - Standard-Images, Baselines, Gruppenrichtlinien / Policies, Host-FW, lokale Admins, Authentifizierung, etc.
- Patch- & Update-Management
 - Betriebssystem, Treiber und Applikationen aktuell halten
- Softwareverteilung
 - Standard-Softwarepakete ausrollen, entfernen, aktualisieren
- Überwachung & Compliance
 - Abweichungen von Vorgaben erkennen (fehlende Patches, falsche Config)

Clients vs. Server – Unterschiede im Endpoint Management

- Nutzungsprofil
 - Clients: stark nutzergetrieben, mobil, wechselnde Netze
 - Server: stabil, 24/7, meist im Rechenzentrum / Cloud
- Patch-Fenster & Downtime
 - Clients: flexibler, Updates auch tagsüber möglich
 - Server: geplante Wartungsfenster, oft strenge Verfügbarkeitsanforderungen
- Standardisierung
 - Clients: viele Gerätetypen, höhere Varianz
 - Server: weniger Typen, stärkere Standardisierung möglich
- Schutzbedarf
 - Clients: oft Einfallstor (Phishing, USB, Browser)
 - Server: oft besonders kritische Daten / Funktionen

Endpoint Management – Herausforderungen

- Heterogene Landschaft (verschiedene OS, Versionen, Hardware)
- Remote-Arbeit & mobile Geräte erschweren zentrale Steuerung
- Fachbereiche installieren „eigene“ Software vorbei an Standards
- Abstimmung von Wartungsfenstern, insbesondere für produktive Server
- Balance zwischen Benutzerfreundlichkeit und strikten Vorgaben

PAM (Privileged Access Management) & PEM (Privileged Endpoint Management)

Begriffe: PAM vs. PEM

- PAM (Privileged Access Management)
 - Verwaltung von hochprivilegierten Konten
 - (Domain-Admins, DB-Admins, Root, Cloud-Admin-Accounts, Service-Accounts)
 - Fokus: Zugang zu kritischen Systemen steuern, absichern und überwachen
- PEM (Privileged Endpoint Management / Privilege Elevation)
 - Verwaltung von lokalen Admin-Rechten auf Endpoints (Windows/macOS/Linux)
 - Fokus: Benutzer arbeiten standardmässig ohne Adminrechte, bekommen bei Bedarf kontrolliert und zeitlich begrenzt erhöhte Rechte

Merksatz:

- PAM schützt Zugänge zu „Privilegierten Systemen & Ports“,
- PEM schützt den Alltag auf Laptops/Desktops/Servern.

PAM: Ziel & Kernfunktionen

- Ziele
 - Missbrauch von Admin-Konten verhindern
 - Passwörter & Secrets (API/SSH/SQL-Keys) zentral verwalten
 - Zugriffe auf kritische Systeme nachvollziehbar machen
- Typische Funktionen
 - Credential Vault / Passwort-Safe: Admin-Passwörter liegen nicht mehr direkt beim Admin, sondern im Tresor
 - Check-out / Just-in-Time Access: Admin holt sich bei Bedarf Zugang (z.B. nur für 1 Stunde), Optional mit Genehmigung („4-Augen-Prinzip“)
 - Session-Recording: Zugriff läuft über PAM-System (RDP/SSH/SQL-Proxy) und Sessions werden mitgeloggt / aufgezeichnet
 - Rotation von Passwörtern / Keys: Automatischer Wechsel nach Nutzung oder in festen Intervallen
 - Beispiele für geschützte Ziele: Domain Controller, Datenbanken, Firewalls, Hypervisor, Cloud-Portale, kritische Linux-Server

PEM: Ziel & Kernfunktionen

- Ziele
 - Enduser ohne dauerhafte Adminrechte arbeiten lassen
 - Trotzdem Support & Entwicklung ermöglichen (Software installieren, Tools ausführen)
 - Angriffsfläche für Malware/Ransomware auf Endpoints reduzieren
- Typische Funktionen
 - Policy-basiertes Rechte-Escalation:
 - z.B.: „User darf diese App als Admin ausführen, aber nicht alles“
 - Regeln nach: Usergruppe, Gerät, App, Signatur, Kontext
 - Just-in-Time Elevation
 - Zeitlich begrenzte Adminrechte, optionale Begründung und Genehmigungs-Workflows (User fordert Elevation an → Admin/Support genehmigt in Konsole)
 - Logging & Reporting: Wer hat wann für welche Aktion erhöhte Rechte bekommen?
- Typische Einsatzorte
 - Entwickler-Laptops, Admin-Workstations, „normale“ Office-Clients, Sprungserver

Vergleich von PAM & PEM

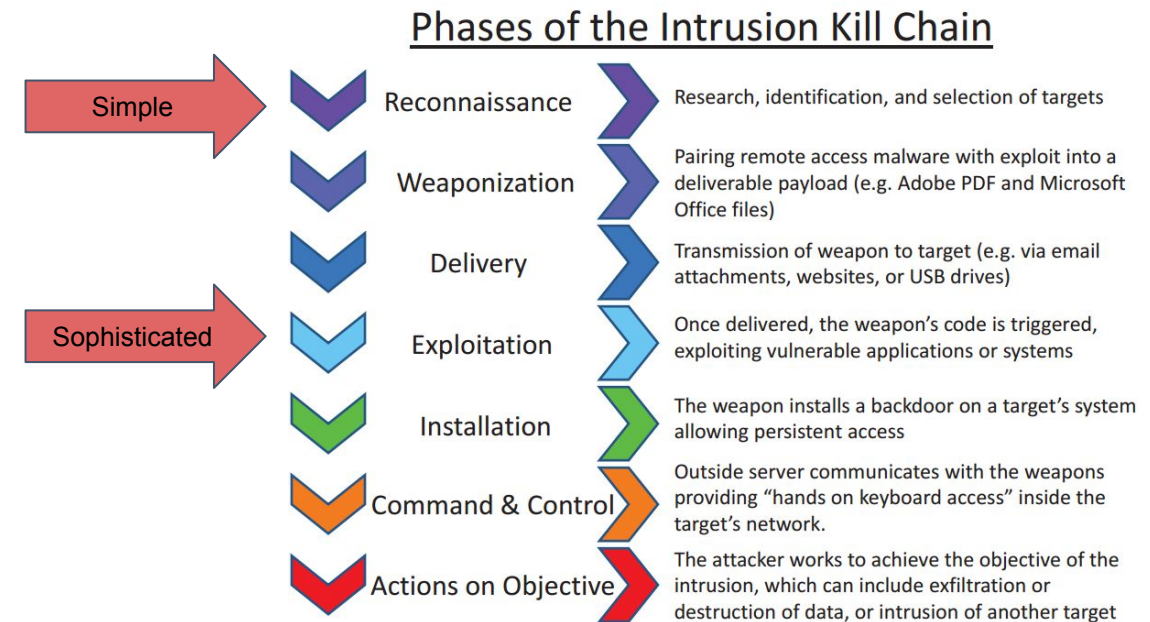
Kriterium	PAM	PEM
Zielobjekte	Server, Datenbanken, Firewalls, Cloud-Portale, „Kronjuwelen“	Endpoints (Clients/Server), lokale Adminaktionen
Konten	Domain-/Root-Admins, Service-Accounts	Normale User, die temporär erhöhte Rechte brauchen
Fokus	Verwaltung & Absicherung von Privileged Accounts	Steuerung von lokalen Elevations auf dem Gerät
Typische Massnahmen	Vault, Session-Proxy, JIT-Access, Passwort-Rotation	App-bezogene Elevation, zeitbegrenzte Adminrechte
Nutzer	zentrale IT-Security, zentrale Admin-Teams	Enduser, Helpdesk, Dev-Teams, lokale Admins

Honeypot und Decoys

Kontext: Kill Chain & Deception

Ziel von Deception (Honeypots/Decoys)

- Angreifer früh entdecken
- Sie verlangsamen / fehlleiten
- Angriffe für Analyse sichtbar machen
- Wirken vor allem in den „inneren“ Phasen der Kill Chain:
 - interne Aufklärung & Lateral Movement (Reconnaissance / Exploitation / Installation)
 - Command & Control und Actions on Objectives
 - Deception fängt Angreifer ab, die bereits einen ersten Fuß im Netzwerk haben.



Was ist ein Honeypot?

- Definition
 - Absichtlich bereitgestelltes, scheinbar attraktives Zielsystem, das in Wahrheit keinen echten Zweck hat
 - Beispiel: „Datenbank-Server“ mit Fake-Daten, „SSH-Server“ für Admins
- Ziele
 - Angreifer anlocken und Aktivitäten beobachten
 - Neue Angriffswege / Tools lernen
 - Frühwarnsystem: „Wer hier reinläuft, ist sehr wahrscheinlich böse / kompromittiert“
- Typen (vereinfacht)
 - Low-Interaction (Simple): emulierte Dienste, wenig echte Funktion
 - High-Interaction (Sophisticated): „echtes“ OS / Dienst, stärker abgeschottet, mehr Einblick

Decoys & Deception Objects

- Decoys = „Verlockende falsche Spuren“, z.B.:
 - Fake-Dateien („Passwörter.xlsx“, „Kronjuwelen-Financen.docx“)
 - Fake-AD-Benutzer (Super User)
 - Fake-Zugangsdaten in Browsern oder Passwort-Managern
 - Fake-SSH-Keys in Download-Folder
 - Fake-Hosts/FQDNs in DNS
 - Fake Server (alter Exchange, SQL-Server, etc.)
- Idee
 - Überall im Netz und auf Geräten kleine Köder verteilen
 - Nur ein Angreifer, der gezielt herumstöbert, wird sie anfassen
 - Jeder Zugriff auf ein Decoy = starker Indikator für Kompromittierung

Deployment Arten

- Externe Decoys:
 - Ködersysteme in DMZ/Cloud um Angriffsvolumen, Quellen und neue Exploits / Kampagnen zu beobachten (Threat Intel)
- Interne Decoys:
 - Als Dateien auf Endgeräten wie Servern & Clients
 - Im Netzwerk als Server, Datenbanken, DNS-Records

Tularosa Studie (2018)

Studie mit >130 Red-Teamern, die in einer Testumgebung angreifen sollten.

Versuchsbedingungen

- mit/ohne echte Decoy-Systeme
- mit/ohne Hinweis „hier könnte Deception aktiv sein“.

Ergebnisse (verkürzt)

- Deception verlangsamt und frustriert Angreifer und erhöht ihren kognitiven Aufwand.
- Schon die bloße Möglichkeit, dass Deception im Spiel ist, verändert Entscheidungen (mehr Vorsicht, mehr Zeitaufwand).

Nutzen & Grenzen von Honeypots/Decoys

- Stärken
- Sehr wenige False Positives
 - Niemand „Legitimes“ sollte Decoy-Zugänge nutzen oder Honeypots scannen
 - Hilft, leise Angreifer aufzuspüren (Aufklärung, Lateral Movement)
 - Liefert wertvolle Threat-Intelligence (Tools, Taktiken, C&C, Exploits)
- Grenzen
 - Deception greift erst, wenn der Angreifer schon intern ist und ersetzt keine Prävention (Patchen, Härtung, EDR, NDR, CNAPP, ...)
 - Decoys müssen isoliert werden, damit Angreifer ihn nicht als Sprungbrett nutzen können
 - Wartung & Realismus: Decoys/Honeypots müssen glaubwürdig wirken (Namen, Daten, Verhalten)

Merksatz: Decoys sind kein Zauber-Schutzschild, aber ein sehr starkes Früherkennungssystem für Angreifer.

IDS (Intrusion Detection System) & IPS (Intrusion Prevention System)

Historischer Kontext & Grundbegriffe

IDS (Intrusion Detection System)

- Erkennt verdächtigen Netzwerkverkehr, löst Alarme aus, blockiert nicht selbst.

IPS (Intrusion Prevention System)

- Sitzt inline im Traffic und kann verdächtige Pakete aktiv blockieren.
- Historie (90er / frühe 2000er)
- Starkes Perimeter-Denken (Innen vs. Außen).
- Viel Traffic war unverschlüsselt (HTTP, Telnet, FTP, Klartext-Mail).
- Idee: „Wir spiegeln den Netzwerkverkehr und suchen darin nach Angriffsmustern.“

Wie arbeiten IDS/IPS?

Signaturbasiert

- Vergleich mit bekannten Angriffsmustern (Snort-Regeln etc.).
- Gut gegen bekannte Exploits, schlecht gegen neue/variierte Angriffe.

Anomaliebasiert

- Erkennen Abweichungen vom „normalen“ Traffic.
- Kann Unbekanntes sehen, aber mehr False Positives.

Platzierung

- Am Rand zum Internet
 - zwischen Router/Firewall und internem Netz
 - überwacht, was rein und raus geht.
- Zwischen Netzsegmenten
 - vor/zwischen Server-Zonen, DMZ, Rechenzentrum
 - überwacht Verkehr zwischen internen Bereichen.

Schwächen vs. moderne Ansätze

- Alles ist verschlüsselt
 - Früher: HTTP, Mail, FTP im Klartext → IPS konnte Inhalt prüfen.
 - Heute: fast alles HTTPS/TLS/QUIC → IPS sieht oft nur „IP + Port“, nicht den Angriff im Payload.
- Kein klarer Perimeter mehr
 - Cloud, SaaS, Homeoffice: Traffic geht direkt vom Client zur Cloud, nicht mehr durch ein zentrales Rechenzentrum.
 - Ein Sensor an einer Stelle sieht nur noch einen Teil des Verkehrs.
- Leistung & Fehlalarme
 - 10/40/100 Gbit/s-Links → Inline-IPS kann zum Flaschenhals werden.
 - Viele Regeln → hoher Pflegeaufwand, sonst zu viele False Positives → Regeln werden oft „entschärft“.
- Stand heute:
 - IDS/IPS sind heute zwar noch in Verwendung, aber ihre Bedeutung hat massiv verloren
 - Viel wichtiger sind EDR, XDR, NDR, SIEM, SSE, CNAPP, WAN-Edge

NDR (Network Detection and Response)

Network Detection & Response (NDR)

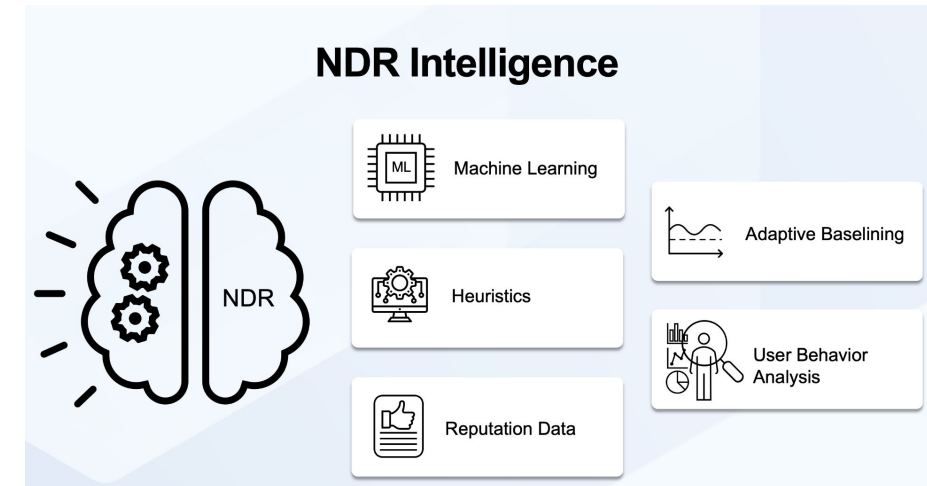
- Kategorie von Security-Lösungen, die
 - den Netzwerkverkehr überwachen,
 - Bedrohungen erkennen
 - und automatisiert oder unterstützt darauf reagieren
- Fokus: Ost-West- und Nord-Süd-Traffic im Netzwerk (on-prem, Cloud, Datacenter)
- Ergänzt Endpoint Detection & Response (EDR) und SIEM/SOC

Motivation für NDR

- Ergänzt Perimeter-Sicherheit (IDS/IPS) für ausgeklügeltere Angriffe:
 - Angreifer bewegen sich lateral im Netz, oft unauffällig
 - Gerade in der Aufklärungsphase um weitere Ziele zu identifizieren
- NDR benötigen keinen Agenten und liefert Sichtbarkeit auf Netzwerkebene:
 - auch für IoT, OT, Server ohne Agent
- Ziel:
 - frühe Erkennung von Anomalien / C&C / Datenabfluss
 - Forensik und Incident Response unterstützen

Funktionsprinzip von NDR

- **Datenquellen:**
 - NDR beruht auf einer möglichst breiten Auswertung von Daten (Layer-2, DNS, NAC, EDR, Proxy, FW-Logs, etc.)
 - Dazu werden Ports und Interfaces von Firewalls & Switchen angebunden (Port-Mirroring, Netflow oder IPFIX)
- **Analyse:**
 - Verhaltens-/Anomalieerkennung (z. B. ML, Baselines)
 - Kontextanreicherung (Assets, User, Geo, Threat Intel)
- **Response:**
 - Alerts ans SOC / SIEM
 - Automatisierte Aktionen:
 - Block/Quarantine via Firewall / NAC / EDR
 - Ticket-Erstellung / SOAR-Playbooks



Was kann NDR erkennen?

- Lateral Movement
 - Ungewöhnliche interne Scans (Port Scans, RDP/SMB zu vielen Hosts)
- Command & Control (C&C)
 - Requests zu auffälligen Domains/IPs
 - Auffällige DNS- oder HTTPS-Muster
- Datenexfiltration
 - Grosse, ungewohnte Transfers nach extern
 - DNS-/HTTP-basierte Tunnels
- Policy-Verstösse
 - Verbotene Protokolle (z. B. SMB nach Internet)
 - Schatten-IT / unbekannte Services

Grenzen und Herausforderungen von NDR

- Verschlüsselter Traffic (TLS, VPN):
 - Inhalte oft nicht sichtbar, Fokus auf Metadaten & Muster
- Hoher Datenumfang: Storage, Rechenlast, Switch-Kapazität, Egress Traffic in Public Clouds kostet Geld! -> Daher hoher Aufwand um Traffic-Streams zu optimieren
- False Positives / Alert-Fatigue, falls schlecht kalibriert
- Hoher Integrationsaufwand:
 - Firewall, EDR, IAM, SIEM, SOAR, etc.
- Kein Ersatz für ssE, Firewall, EDR oder Netzwerksegmentierung, sondern zusätzlicher Baustein im SOC für sehr hohe Compliance Anforderungen
- Eher klassisches, On-Prem fokussiertes Produkt, dass Probleme mit remote Clients, Public-Clouds und SASE-Frameworks hat (vor allem auch, weil diese ähnliche Logiken in ihren Produkten bereits integrieren)

EDR (Endpoint Detection & Response)

EDR Evolution

- Antivirus (AV, 1980 - 2000er)
 - Signaturbasierter Schutz auf dem Endpoint
 - Erkannt werden vor allem bekannte Malware-Varianten
 - Kein weiterer Kontext, keine Verlaufsdaten (keine „Telemetrie“, also aufgezeichnete Ereignisse auf dem Endpoint)
- Next-Gen AV / Endpoint Protection (EPP, Mitte 2000er-2010er)
 - Ergänzt Signaturen um Heuristiken, Verhaltensanalysen, Exploit-Schutz
 - Ziel: Angriffe möglichst früh blocken (Prävention)
 - Fokus weiterhin: Schutz vor der Ausführung, wenig Sicht auf das „Danach“
- EDR (Endpoint Detection & Response, ab 2012+)
 - Kontinuierliche Endpoint-Daten (Prozesse, Dateien, Logins, Netzwerk vom Host) werden aufgezeichnet
 - Erkennung von komplexeren Angriffen, Threat Hunting, Forensik & gezielte Response auf dem Endpoint
 - Ergänzt EPP – Prävention + Detektion + Reaktion

EDR: Ziel & Grundidee

- Auf jedem Rechner/Server läuft ein Agent (kleine Software).
- Dieser Agent beobachtet den Rechner:
 - Welche Programme laufen?
 - Welche Dateien werden geändert?
 - Wohin baut der Rechner Verbindungen auf?
 - Die Daten gehen an eine zentrale EDR-Plattform (meist in der Cloud).
- Ziel von EDR:
 - Angriffe erkennen, auch wenn kein klassischer Virus im Spiel ist.
 - Den Ablauf eines Angriffs nachvollziehen.
 - Den betroffenen Rechner gezielt „aufräumen“ oder isolieren.

EDR Features: Sichtbarkeit & Erkennung

- Prozesse / Programme
 - Welche Programme starten? Wer startet wen? (z.B. Word → PowerShell)
- Dateien & System
 - Welche Dateien werden erstellt, geändert, verschlüsselt?
 - Änderungen an Registry / Autostarts / Diensten
- Netzwerk vom Host
 - Welche Programme sprechen mit dem Internet?
 - Ungewöhnliche Ziele (z.B. seltene IPs, komische Domains)
- Benutzer-Aktivität
 - Logins / Logouts
 - Rechteänderungen (z.B. plötzliche Admin-Rechte)
- EDR benutzt das für:
 - Regelbasierte Erkennung: „Wenn X und Y passieren → verdächtig“
 - Verhaltensanalyse: „Dieses Verhalten passt nicht zu diesem Rechner/Benutzer“

EDR Features: Prävention & Hardening

- Sandbox-Analyse
 - Verdächtige Dateien werden in einer abgeschotteten Umgebung gestartet.
 - Die EDR-Plattform schaut: verhält sich die Datei wie Malware?
 - Ergebnis: Datei blocken / erlauben
- USB- / Geräte-Kontrolle
 - Regeln für USB-Sticks & Hardware:
 - z.B. „USB nur lesen“, „nur verschlüsselte Sticks“, „keine unbekannten Geräte“.
- Application Control
 - Welche Programme dürfen überhaupt laufen?
 - z.B. nur erlaubte Business-Apps, alles andere blocken → Whitelisting.
- Endpoint-Firewall / Netzwerk-Kontrolle
 - Welche Programme dürfen ins Netz?
 - Blockieren bestimmter Ports/Protokolle direkt am Host.

EDR Features: Response, Hunting & Automation

- Automatische Aktionen
 - Verdächtigen Prozess beenden
 - Datei in Quarantäne verschieben oder löschen
 - Autostart-Einträge / Persistenz entfernen
 - Rechner isolieren: Kein normales Netzwerk mehr, nur noch Verbindung zur EDR-Konsole
- Untersuchung & Hunting
 - Nachschauen: „Wo ist diese Datei / Domain / IP noch aufgetaucht?“
 - Zeitstrahl („Timeline“) des Vorfalls sehen:
 - Phishing-Mail → Anhang → Prozesskette → C&C → Exfiltration
 - Analysten können aktiv suchen:
 - „Zeig mir alle Rechner, wo powershell.exe mit auffälligem Kommando lief.“
- Automatisierung
 - Playbooks (SOAR = vorgebaute Reaktionspläne) wie:
 - „Wenn Ransomware-Verdacht → Host isolieren + Ticket erstellen + Mail ans SOC“.

EDR Stärken, Grenzen & Einordnung

- Stärken
 - Sieht viel mehr als ein klassischer Virens Scanner (nicht nur Dateien), u.a:
 - fileless Angriffe (nur Scripts, PowerShell, LOLBins)
 - Laterale Bewegung (z.B. Tools für Netzwerkausbreitung)
 - Unterstützt Forensik & Threat Hunting:
 - Retrospektives nachvollziehen, was wirklich passiert ist.
 - Sehr gezielte Massnahmen möglich, z.B.: Einen einzelnen Host isolieren
- Grenzen
 - Braucht einen Agenten auf dem Gerät: schwierig bei IoT, OT, alten Systemen (wohingegen ein DNR alles sieht)
 - Viele Daten → ohne gutes Tuning: zu viele Alarme.
 - Datenschutz/Privacy: EDR sieht, was Benutzer auf dem Rechner machen.

EDR Erweiterung & Ausblick

- Von EDR zu XDR
 - EDR-Daten werden mit weiteren Quellen kombiniert (Mail-Security, Proxy, NAC, NDR, Identity, Cloud-/ssE-Dienste).
 - Diese zusammengeführte Detection- & Response-Plattform nennt man XDR (Extended Detection & Response).
- Betriebsmodelle: intern vs. extern
 - EDR/XDR können durch ein internes SOC betrieben werden – oder durch einen externen Dienstleister.
 - Wird die Überwachung und Reaktion von Menschen (24/7 Dienstleister) als Service übernommen, spricht man von MDR (Managed Detection & Response).
- Rolle von SIEM & SOC
 - EDR/XDR liefern „hochwertige“ Security-Events, die oft zusätzlich in ein SIEM eingespeist werden.
 - Das SIEM sammelt Logs aus vielen Systemen und dient dem SOC als zentrale Analyse-, Korrelation- und Reporting-Plattform.
- Praxis: kleine vs. grosse Unternehmen
 - Für viele KMU reicht eine gut betriebene EDR/XDR-Lösung, oft zusammen mit einem MDR-Service, aus.
 - Grosse Unternehmen setzen meist auf eine Kombination aus SIEM + XDR + eigenem SOC, haben dafür aber auch Budget und Spezial-Know-how.

Vergleich NDR und EDR

Kriterium	EDR (Endpoint Detection & Response)	NDR (Network Detection and Response)
Perspektive	Blick in einen einzelnen Rechner / Server	Blick auf das Netzwerk und den Verkehr zwischen Systemen
Datenquellen	Agent auf dem Endpoint: Prozesse, Dateien, Logins, lokale Verbindungen	Netzwerkmitschnitte (SPAN/TAP), Flows, DNS-/Proxy-/Firewall-Logs
Typische Erkennungen	Malware, verdächtige Programme, „Living off the Land“ auf dem Host, verdächtige Prozessketten (z.B. Office → PowerShell)	Ungewöhnliche interne Scans, verdächtige Verbindungen (C2), grosse/ungewohnte Datenübertragungen, DNS-/HTTP-Tunnel, Policy-Verstösse (z.B. SMB ins Internet).
Reaktionsmöglichkeiten	Host isolieren, Prozess beenden, Datei in Quarantäne	Verbindungen blocken, Netzsegment isolieren, Firewall/NAC-Regeln anpassen
Stärken	Tiefe Sicht auf einen Host, sehr gezielte Massnahmen möglich	Gute Übersicht über das Netz, sieht auch Geräte ohne Agent (IoT/OT/BYOD)
Grenzen	Funktioniert nur dort, wo ein Agent installiert ist.	Sieht nur Netzwerk-Verkehr, keine Prozesse auf dem Host; verschlüsselter Traffic (TLS/VPN) erschwert Analyse; braucht Zugriff auf relevante Netzwerk-Segmente.

SIEM (Security Information and Event Management)

SIEM: Ziel & Grundidee

- **Sammelt Logs und Events** aus vielen Systemen an einem Ort:
- Firewalls, Server, Anwendungen, Datenbanken, AD/Identity, EDR/NDR, Cloud, etc.
- Speichert, korreliert und visualisiert diese Daten.
- Ziele von SIEM:
 - Sicherheitsvorfälle erkennen
 - Zusammenhänge zwischen Events aus verschiedenen Systemen sehen
 - Nachvollziehen, was wann wo passiert ist (Forensik)
 - Reports für Compliance & Audits liefern

SIEM: Datenquellen & Architektur

- **Datenquellen** (Log-Quellen)
 - Netzwerk: Firewalls, Proxys, NDR, VPN
 - Systeme: Windows-/Linux-Logs, AD, Datenbanken, Webserver
 - Security-Tools: EDR/XDR, IDS/IPS, AV, Mail-Security
 - Cloud & SaaS: z.B. Azure AD, Office 365, AWS, GCP
- **Datenpipeline**
 - Logs werden per Syslog, Agent oder API ans SIEM geschickt
 - SIEM normalisiert die Daten (einheitliches Format/Fields)
 - Speicherung im Log-Store / Data Lake
- **Analyse & Oberfläche**
 - Korrelation über viele Quellen
 - Dashboards, Suchabfragen, Alarme, Reports
- **Wichtig:** Ein SIEM sieht vieles, aber nur, was man ihm an Logs liefert.

SIEM Use-Cases

- Realtime-Überwachung & Alarme
 - Verdächtige Logins (z.B. viele Fehlschläge, Login aus ungewöhnlichem Land)
 - Kombination von Events (z.B. neuer Admin-User + Firewall-Änderung)
- Korrelation & Angriffsketten
 - Events aus verschiedenen Systemen zu einer Story verbinden:
 - Phishing-Mail → ungewöhnlicher Login → neue Prozesse → Datenabfluss
- Threat Hunting & Forensik
 - Suche: „Zeig mir alle Hosts, wo dieser User in den letzten 30 Tagen eingeloggt war.“
 - Nachträgliche Analyse nach einem Incident
- Compliance & Reporting
 - Nachweis: „Wer hat wann worauf zugegriffen?“
 - Standard-Reports (ISO, PCI-DSS, NIS2, ...)
- SIEM ist vor allem ein Analyse- und Suchwerkzeug – kein Produkt, das von alleine alles löst.

SIEM: Stärken & Grenzen

- Stärken
 - Zentrale Sicht auf sehr viele Systeme
 - Gut für Forensik, Audits und Compliance
 - Kann komplexe Korrelationen über lange Zeiträume machen
 - Basis-Werkzeug für ein SOC (Security Operations Center)
- Grenzen
 - Sehr aufwändig zu Implementieren und Betreiben:
 - Onboarding der Log-Quellen, Normalisierung, Regelpflege
 - Speicher & Rechenleistung (viele Logs)
 - Bilden von Korrelationen und Alerts
 - Ohne gutes Tuning gibt es viele Alarme aber wenig Erkenntnis und eine Abstumpfung gegenüber Alarmen
 - Inhalt der Logs ist sehr technisch, die Interpretation braucht erfahrene Security Analysten
 - Macht selbst keine aktive Response – dazu braucht man weitere Tools (XDR oder SOAR)

SIEM im Zusammenspiel mit EDR, XDR & SOC

- Mit EDR / XDR / NDR
 - EDR/XDR/NDR erzeugen hochwertige Security-Events
 - Diese können ins SIEM eingespeist und mit anderen Logs kombiniert werden
 - Beispiel: EDR meldet verdächtiges Verhalten → SIEM prüft: “Gab es vorher ungewöhnliche Logins / VPN-Verbindungen / Mail-Events?”
- Rolle im SOC
 - Das SIEM ist die zentrale Konsole für ein SOC indem Monitoring, Suche, Fall-Dokumentation, Compliance-Checks, Forensik und Reporting ausgeführt wird
 - SIEMs können aber selbst keine Response auf einen Vorfall durchführen, sondern lösen solche über Integrationen mit SOAR oder XDR-Tools dann wiederum aus.
- Verwendung in Unternehmen
 - Für viele KMU: XDR/MDR ist häufig genug.
 - In grossen Organisationen trifft man häufig auf: SIEM + XDR + eigenes / managed SOC

SOAR - Security Orchestration, Automation & Response

SOAR – Security Orchestration, Automation & Response

- Software-Plattform, die:
 - Security-Tools verbindet (SIEM, EDR/XDR, NDR, Mail-Security, Firewalls, Ticketing, ...)
 - wiederkehrende Aufgaben automatisiert
 - Playbooks für Incident Response ausführt
- Wird oft “on top” auf ein SIEM gebaut, um SOC-Prozesse zu automatisieren, kann aber auch ohne SIEM direkt mit anderen Komponenten integriert werden (EDR, Proxy, Mail, FW, etc.)
- Ziel:
 - schnellere und konsistente Reaktion auf Sicherheitsvorfälle
 - Entlastung des SOC (weniger Klickarbeit, weniger Copy-Paste zwischen Tools)
- Merksatz:
 - SIEM findet Dinge, SOAR erledigt die Routinearbeit rund um die Reaktion.
 - SOAR ersetzt kein SIEM und kein XDR – es orchestriert die Reaktion darauf

Bausteine eines SOAR-Systems

- Integrationen / Konnektoren
 - Schnittstellen zu: SIEM, EDR/XDR, NDR, AV, Mail-Gateway, Firewalls, AD, Ticket-System, ...)
 - SOAR kann dort lesen und schreiben (Aktionen anstossen)
- Playbooks (Runbooks)
 - Grafische oder skriptbasierte Workflows
 - Beschreiben Schritt für Schritt: „Wenn Alarm X → tue A, B, C...“
- Case Management
 - Jeder Incident wird als Fall/Ticket geführt
 - Alle Aktionen, Notizen, Artefakte werden zentral dokumentiert
- Automatisierung + Mensch im Loop
 - Vollautomatische Schritte (z. B. IOC-Check (Indicator of Compromise), Enrichment)
 - Entscheidungen, bei denen ein Analyst klicken/bestätigen muss (z. B. „Diesen User wirklich sperren?“)

Beispiel: Phishing-Fall mit mehreren Systemen

- Phishing-Mail

Mail-Security: Phishing Mail kommt durch, wird aber als „verdächtig“ geloggt.

- Kontoübernahme

- User klickt → Login auf Fake-Seite → IdP meldet ungewöhnliches Login (neues Gerät/Land).

- Endpoint-Befall & Ausbreitung

- Angreifer nutzt Konto, schickt interne Mails mit Anhang → EDR auf einem Laptop erkennt verdächtige Prozesskette (Office → PowerShell → C&C).

- Korrelation & Reaktion

- SIEM verknüpft: verdächtige Mail + auffälligen Login + EDR-Alarm → High-Risk-Incident.
- SOAR-Playbook: Konto sperren/MFA erzwingen, Host isolieren (XDR), Mails in Quarantäne, Ticket + Info ans SOC.

SOAR: Zusammenspiel mit SIEM, EDR, NDR & SOC

- SOAR kann in drei typischen Varianten betrieben werden:
- 1) SOAR auf Basis eines SIEM (klassisches SOC)
 - SIEM sammelt Logs & korreliert Events aus vielen Quellen
 - SOAR erhält die korrelierten Alarme und startet Playbooks
 - Geeignet für grosse Umgebungen / eigenes SOC mit vielen Use Cases
- 2) SOAR auf Basis eines XDR
 - XDR korreliert bereits Security-Telemetrie (Endpoint, Mail, Identity, Cloud, ggf. Netzwerk)
 - SOAR bekommt Alarme direkt vom XDR und führt weitergehende Workflows aus
 - Häufiges Modell bei modernen Umgebungen / MDR-Providern
- 3) SOAR direkt an Security-Tools angebunden
 - SOAR spricht direkt mit EDR, NDR, Firewalls, SSE, Mail-Security, Identity, ...
 - Automatisiert gezielt einzelne Abläufe (Host isolieren, User sperren, Mail löschen etc.)
 - Weniger komplexe Korrelation, dafür einfacher zu bauen → gut für kleinere Unternehmen

Stärken und Schwächen von SOAR

- Stärken
 - Entlastet das SOC von Routineaufgaben
 - Reaktion wird schneller und einheitlicher (Playbooks statt „jeder macht es anders“)
 - Gute Dokumentation aller Schritte im Incident
 - Erlaubt 24/7-„Grundreaktion“, z.B. wenn das SOC kein 24/7 Dienst hat
- Grenzen
 - Braucht ein funktionierendes Ökosystem (SIEM/XDR/Tools + gute Use-Cases)
 - Sehr hoher Aufwand und Komplexität für:
 - Integrationen
 - Playbooks (designen, testen, pflegen)
 - Schlechte oder zu aggressive Playbooks können:
 - Dinge kaputtmachen („falscher User gesperrt“, „falscher Host isoliert“)
 - SOAR erkennt keine Angriffe – es reagiert nur auf Signale anderer Systeme

SOC (Security Operations Center)

SOC: Ziel & Grundidee

- Team + Prozesse + Tools zur Überwachung und Reaktion auf Sicherheitsvorfälle
- Stellt die zentrale operative Einheit für Security Detection & Response da
- Üblicherweise vor allem in grossen und stark reglementierten Unternehmen anzutreffen
- Typische Ziele:
 - Sicherheitseignisse früh erkennen
 - Reagieren & koordinieren, wenn etwas passiert
 - Überblick über die Gesamtlage behalten (Lagebild)
 - Nutzt verschiedene Systeme als Tools wie: SIEM, XDR/EDR, NDR, SOAR, Mail-Security, FW, SSE
- **Wichtig:** Ein SOC ist kein Tool, sondern eine Organisatorische Einheit

Aufgaben eines SOC

- **Monitoring & Triage**
 - Auswertung von Monitoring-Dashboards (SIEM, XDR, SOAR)
 - Alarme bewerten: „Fehlalarm oder echter Vorfall?“
- **Incident Response**
 - Vorfälle klassifizieren (Severity, Impact)
 - Maßnahmen anstossen: Host isolieren, User sperren, Regeln anpassen, ...
 - Kommunikation mit IT, Fachabteilungen, ggf. Management
- **Threat Hunting & Verbesserung**
 - Proaktiv nach verdächtigen Mustern suchen
 - Use Cases & Erkennungsregeln verbessern (SIEM/XDR-Tuning)
 - Lessons Learned & Playbooks erstellen und aktualisieren
- **Reporting & Compliance**
 - Berichte für Management, Revision, Audit
 - KPIs (z.B. Time to Detect, Time to Respond)

SOC: Rollen & Aufbau

- **Tier-1 Analyst** (Alert Handler)
 - Erstes Screening der Alarme
 - Einfache Checks & Eskalation bei echten Vorfällen
- **Tier-2 Analyst** (Incident Responder)
 - Detaillierte Analyse, Korrelation, Ursachenfindung
 - Plant und koordiniert Gegenmassnahmen
- **Tier-3 / Spezialisten**
 - Threat Hunter, Forensiker, Malware Analysten
 - Entwickeln neue Use Cases, tiefe Analysen
- **SOC Lead / Manager**
 - Verantwortlich für Prozesse, Prioritäten, Reporting
 - Schnittstelle zu CISO / Management

SOC: Betriebsmodelle

- **Internes SOC**
 - Eigenes Team, eigene Infrastruktur
 - Hohe Kontrolle, viel Know-how nötig, äusserst teuer
- **Externes SOC**
 - Betrieb durch Dienstleister (Managed Detection & Response)
 - Dienstleister nutzt meist eigenen XDR/SIEM/SOAR-Stack
 - Gut für kleinere/mittlere Unternehmen die aufgrund von Compliance-Anforderungen ein SOC benötigen
- **Hybrides SOC**
 - Separierung auf Basis von:
 - Tiers
 - Betriebszeiten
 - Use Cases
 - Für Unternehmen, die die Kosten für ein externes SOC senken wollen und einzelne Tätigkeiten selbst übernehmen

Adversarial Exposure Validation (AEV)

Worum geht es bei AEV

- Kontinuierliche, angreiferorientierte Sicherheitstests
 - simulieren echte Angreifer-TTPs statt nur Schwachstellen zu „scannen“
- Ziel: validieren, ob identifizierte Exposures (Vulns, Misconfigs, schwache Identities, Pfade) im realen Umfeld wirklich ausnutzbar sind
- Typische Quellen für Exposures:
 - Vulnerability-Scanner, CNAPP, ASM, IAM-Daten, manuelle Findings
- Kernfragen:
 - Kann ein Angreifer diese Lücke praktisch ausnutzen?
 - Welche Attack-Paths ergeben sich daraus?
 - Welche Security Controls bremsen oder verhindern die Ausnutzung?
 - Wie hoch ist der tatsächliche Business Impact?
- AEV ist ein Ansatz / Rahmen, der Breach and Simulation (BAS), automatisiertes Pentesting und (teilweise) Red-Teaming zusammenführt

Wie funktioniert AEV?

- Threat-informed Szenarien
 - Auswahl relevanter Angriffs-Szenarien auf Basis von Threat Intel & MITRE ATT&CK
 - Fokus auf realistische TTPs statt synthetischen Tests
- Automatisierte Angriffssimulationen
 - Werkzeuge emulieren externe und interne Angreifer
 - testen z.B.: Initial Access, Lateral Movement, Credential Abuse, Exfiltration
- Validierung von Exposures & Controls
 - Prüft, ob gemeldete Schwachstellen/Fehlkonfigurationen wirklich zum Ziel führen
 - bewertet gleichzeitig, ob Prevention-/Detection-Controls wirksam anschlagen
- Kontinuierlich & skaliert
 - nicht nur 1× im Jahr wie klassischer PenTest,
 - sondern laufende, automatisierbare Testzyklen → passt gut in CTEM-Programme

Stärken & Schwächen von AEV

Stärken

- **Realitätsnah:** Testet „wie ein Angreifer“ und zeigt, welche Exposures wirklich ausgenutzt werden können
- **Risikobasiert:** Priorisiert nicht nur nach CVSS (Common Vulnerability Scoring System), sondern nach tatsächlicher Exploitierbarkeit + Impact
- **Kontinuierliche Sicherheitsvalidierung:** schliesst die Lücke zwischen punktuellen PenTests und reiner Schwachstellensammlung

Schwächen / Herausforderungen

- **Reifegrad nötig:** Erfordert klares Scoping, gute Change-/Freigabeprozesse und Monitoring – sonst Risiko für Störungen
- (Noch) **kein Ersatz für andere Technologien:** Ergänzt klassisches Vulnerability Management, PenTests und Red-Teaming, ersetzt sie aber nicht komplett -> Entwicklung geht aber in diese Richtung
- **Tool- & Datenabhängig:** Qualität hängt stark von verfügbaren Exposure-Daten und der Güte der Angriffssimulationen ab

AEV im Kontext

- Vulnerability Scanning
 - Findet technische Schwachstellen (CVEs, Versionsstände).
- CNAPP
 - Findet Cloud-bezogene Exposures
 - (Fehlkonfigurationen, schwache Workloads, Rechte, plus auch Schwachstellen).
- AEV
 - Testet wie ein Angreifer, ob diese Exposures wirklich ausnutzbar sind
 - Und welche Pfade/Controls dabei relevant sind.
- CTEM
 - Ist das Gesamtprogramm, das alle diese Quellen nutzt, um kontinuierlich zu entscheiden:
 - Wo bin ich wie exponiert?
 - Was ist wirklich kritisch?
 - Was wird wann von wem behoben?
 - Sind meine Massnahmen wirksam?

Exposure Validation vs Red-Teaming

Kriterium	Exposure Validation	Pentest / Red-Teaming
Zielsetzung	Kontinuierliche Validierung tatsächlich ausnutzbarer Schwachstellen	Tiefgehende Analyse der gesamten Sicherheitslage
Frequenz	Regelmässig oder kontinuierlich	Punktuell (Produktentwicklung, Audits)
Methodik	Automatisierte, bedrohungsbasierte Simulation (z. B. MITRE ATT&CK)	Manuelle, kreative Angriffsszenarien inkl. Social Engineering
Automatisierung	Hoch	Gering
Ergebnisdauer	Schnell, oft in Echtzeit	Langsam, oft mehrere Tage bis Wochen

Password Manager

Enterprise Passwort-Manager: Features

- **Sicherer Tresor** pro Nutzer: Ende-zu-Ende verschlüsselte Vaults für Passwörter, Passkeys, sichere Notizen, Dateien.
- **Starke Credentials & Komfort**: Passwort-Generator, Auto-Fill im Browser & in Apps, Sync über alle Geräte.
- **Team-Sharing & Rollen**: Gemeinsame Ordner für Teams, fein granulierte Rechte (view / edit / share), Rollen & RBAC für Admin-Funktionen.
- **Integration ins IAM**: SSO-Anbindung an Identity Provider mit automatischem Provisioning für Nutzer und Gruppen
- **Auditing**-Funktionen: Passwort-Health, Alerts, detaillierte Audit-Logs, SIEM-Integration, etc.
- Vielzahl von Erweiterungen:
 - Dark-Web / Breach-Monitoring
 - Secrets-Management
 - Privileged-Access-/Endpoint-Privileg-Module

Backup

Grundidee und KPIs

- **Backups: Kopie von Daten, die getrennt vom Original aufbewahrt wird**
- Ziel:
 - Datenverlust durch Fehler (Hardware, Software, Mensch) abfangen
 - Datenverlust durch Angriffe (z.B. Ransomware) überleben
 - Wiederherstellung von Systemen und Dateien ermöglichen
- Grundideen
 - Backup ist kein Archiv:
 - Backup = „Ich will den Zustand von gestern wiederhaben“
 - Archiv = „Ich will wissen, was vor 5 Jahren drin stand“
- Zwei wichtige Begriffe (KPIs):
 - RPO (Recovery Point Objective): Wie viel Datenverlust ist akzeptabel? (z.B. max. 4 Stunden)
 - RTO (Recovery Time Objective): Wie schnell muss es wieder laufen? (z.B. max. 2 Stunden)

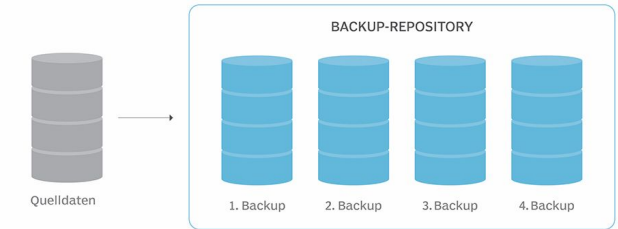
Klassische Backup-Konzepte

- Klassische Backup-Konzepte
- Backup-Typen
 - Voll-Backup: alles (Snapshots)
 - Inkrementell: nur Änderungen seit letztem Backup
 - Differenziell: Änderungen seit letztem Voll-Backup
- Medien
 - Früher: Bandlaufwerke (Tape) – langsam, aber billig und gut lagerbar
 - Heute: Disk, Appliance, Cloud-Storage, oft Kombination
- Grundprinzip „3-2-1-Regel“
 - 3 Kopien der Daten
 - 2 verschiedene Medientypen
 - 1 Kopie an einem anderen Ort (z.B. anderes Rechenzentrum, Cloud)
- Ziel: Auch wenn ein Rechenzentrum brennt oder verschlüsselt wird, gibt es noch eine brauchbare Kopie.

Volles, inkrementelles und differentielles Backup im Vergleich

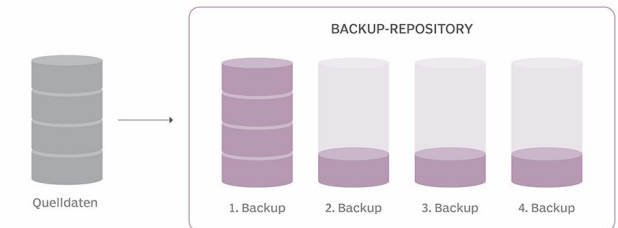
Voll-Backup

Die Daten werden jedes Mal vollständig kopiert.



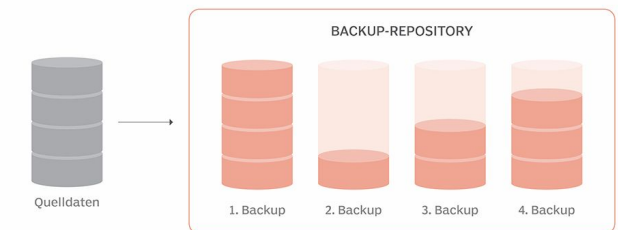
Inkrementelles Backup

Die Daten werden zunächst vollständig kopiert, und danach werden bei jedem weiteren Backup nur neue oder aktualisierte Daten gesichert.



Differentielles Backup

Die Daten werden zunächst in ihrer Gesamtheit kopiert, und danach werden bei jedem weiteren Backup nur die Backup-Sets mit einer Änderung gesichert.



Backup-Architekturen

- Agent-basiert vs. agentlos
 - Agent-basiert: kleine Software auf dem System → sieht mehr (Transaktionsstatus, DB, offene Dateien)
 - Agentlos: Backup-Server nutzt APIs / Protokolle (z.B. Hypervisor, M365-API, SMB, NFS) → weniger Installationsaufwand
- Pull- vs. Push-Modell
 - Pull: Backup-Server stößt Sicherung an („zieht“ Daten vom Client) – klassisch bei Servern/VMs
 - Push: Client stößt Sicherung an („schiebt“ Daten, z.B. Laptop-Backup über Internet)
- Backup-Topologien
 - Nur lokales Backup-Repository (Disk/Appliance im gleichen RZ): Schnelles Restore, aber kaum Schutz bei RZ-Ausfall / Ransomware
 - Zwei-Stufen-Architektur (heute typisch): Primär-Backup: lokales Repository für schnelle Restores und Sekundär-Backup zur Replizierung in zweites RZ oder Cloud (Desaster-Schutz & zusätzliche Ransomware-Resilienz)
- On-Prem vs. Backup-as-a-Service
 - On-Prem: Backup-Software & Storage im eigenen RZ
 - BaaS: Backup-Logik und oft auch Storage als Cloud-Service, Anbindung über Agenten/APIs/VPN/Direct Connect

Moderne Backup- und Data-Protection-Plattformen

- Machen viel mehr als „nur dummes Kopieren“: Backup-Software + Cloud-Speicher + Data-Protection
- Intelligente Speicherung
 - Deduplizierung & Kompression → weniger Speicherbedarf
 - Ziele: lokaler Storage, zweites Rechenzentrum, Cloud / Object Storage
- Anwendungs- & Cloud-Verständnis
 - Spezielle Integrationen für VMs, Datenbanken, Fileshares, Kubernetes, M365, SaaS
 - Konsistente Snapshots statt „wildes Datei-Kopieren“ inkl. Restore von einzelnen Dateien / Datenbanken / Ordnern
- Data Protection & Governance
 - Immutability: Sicherungen sind für einen Zeitraum nicht änder- oder löscherbar
 - Aufbewahrungsregeln (Retention) für unterschiedliche Datenklassen / Compliance-Vorgaben
- „Smart Features“
 - Zentrale Suche / Index über gesicherte Daten
 - Erkennung sensibler Daten (z.B. personenbezogen)
 - Erkennen ungewöhnlicher Muster (z.B. plötzlich sehr viele geänderte Dateien)
 - Malware-Suche & Entfernung

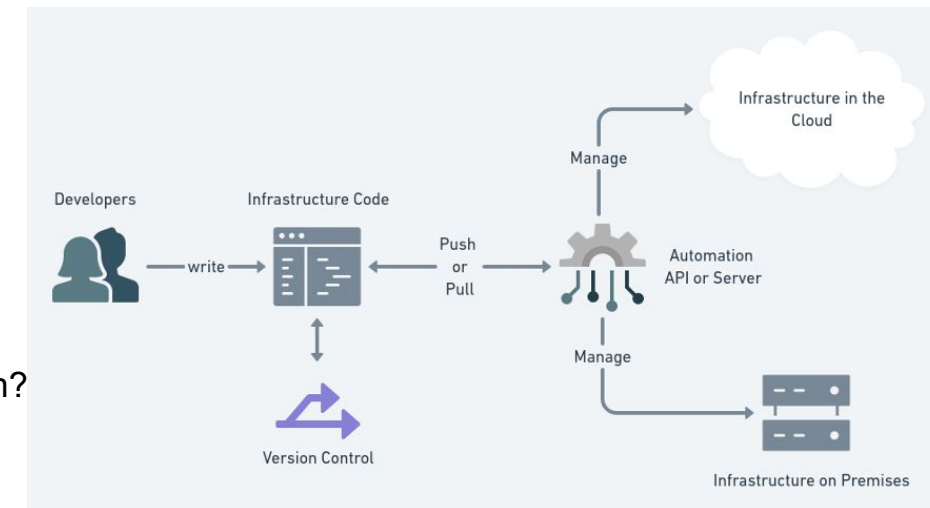
Backup Angriffe und Schutzkonzepte

- Typische Angriffe auf Backups
 - Ransomware verschlüsselt Produktivsysteme und Backup-Repositories (nach “Einnisten”)
 - Angreifer übernehmen Backup-Admin-Konten und löschen Jobs / Kataloge
 - Manipulation von Snapshots
- Kern-Schutzmechanismen
 - Immutable Backups: für definierte Zeit nicht löscher oder veränderbar – selbst durch Admins
 - Trennung / Air-Gap: Kopien in anderer Umgebung, anderem Account oder physisch getrennt
 - Strenges Rechtekonzept + MFA für Backup-Admins, getrennt vom normalen AD / Identity
- Anomalie-Erkennung im Backup-System
 - ungewöhnlich viele Änderungen
 - auffällige Failures / gelöschte Jobs
- Regelmässige Restore-Tests (Disaster Recovery & Business Continuity):
 - nicht nur „Backup-Job grün“, sondern tatsächliche Wiederherstellung (automatisiert) testen
 - definierte RTO/RPO realistisch prüfen

IaC (Infrastructure as Code)

Infrastructure as Code – Konzept und Definition

- Definition: IaC bezeichnet den Ansatz, IT-Infrastruktur (z.B. Netzwerke, virtuelle Maschinen, Datenbanken) in maschinenlesbaren Beschreibungen (Code) zu definieren
- Zentrale Merkmale:
 - Infrastruktur wird in Dateien (z.B. HCL, YAML) spezifiziert.
 - Diese Dateien werden wie Software in Versionskontrollsystemen (z.B. Git) verwaltet.
 - Änderungen an der Infrastruktur erfolgen durch Änderungen am Code, nicht durch direkte Konfiguration in GUIs.
- Deklarativ vs. imperativ:
 - Deklarativ: beschreibt den gewünschten Zielzustand („wie soll es am Ende aussehen?“)
 - Imperativ: beschreibt die Abfolge von Schritten („wie komme ich dahin?“).
 - Viele IaC-Tools folgen einem deklarativen Ansatz.
- Zielsetzung:
 - Reproduzierbarkeit und Konsistenz von Umgebungen
 - Nachvollziehbarkeit von Konfigurationsänderungen
 - Integration von Infrastruktur in Software-Engineering-Prozesse (DevOps)



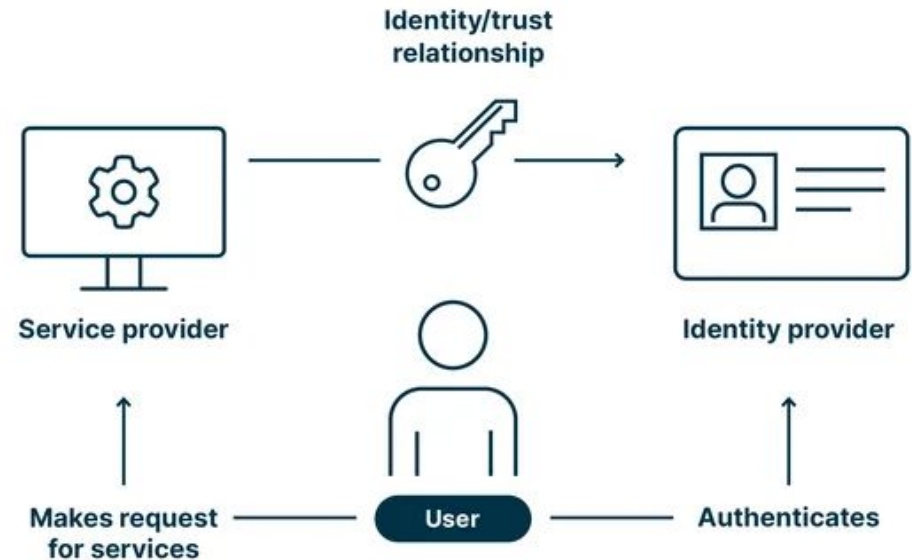
Vorteile und Nachteile von IaC

Vorteile	Nachteile
Nachvollziehbarkeit & Transparenz	Komplexität bei grossen Umgebungen
Skalierbarkeit	Drift-Gefahr
Reproduzierbarkeit	Erhöhter Lernaufwand für Teams
Fehlerreduktion	Fehlerauswirkung
Unterstützt Devops-Ansatz	Abhängigkeit von Tools und Versionen

Identity Protection

Digitale Identität im Netzwerk

- Digitale Identität = Menge von Attributen, die ein Subjekt eindeutig beschreiben
 - z. B. Benutzername, Matrikelnummer, E-Mail, Rollen (Student, Admin ...)
- Subjekte:
 - Menschen (Nutzerkonten)
 - Systeme/Services (Server, Microservices, APIs)
 - Geräte (Clients, IoT)
- Zentrale Begriffe
 - Authentisierung: „Wer bist du?“ – Überprüfung der Identität
 - Autorisierung: „Was darfst du?“ – Rechte/Zugriffsentscheidung
 - Credentials: Nachweise der Identität (Passwort, Token, Zertifikat, Hardware-Token)



Bedrohungen für digitale Identitäten

- Passwortbezogene Angriffe
 - Schwache / wiederverwendete Passwörter
 - Brute-Force & Wörterbuchangriffe
 - Credential Stuffing (wiederverwenden geleakter Logins)
- Social Engineering & Phishing
 - Nutzer wird manipuliert, sein Passwort oder Einmalcode preiszugeben
 - Täuschend echte Login-Seiten, gefälschte E-Mails
- Technische Angriffe
 - Session Hijacking: Session-Cookies werden gestohlen/missbraucht
 - Man-in-the-Middle: Kommunikation wird mitgelesen/manipuliert
 - Missbrauch von gestohlenen Tokens, API-Keys, Zertifikaten
- Folgen
 - Identitätsdiebstahl, Kontenübernahme
 - Zugang zu internen Systemen, Datenabfluss, Sabotage

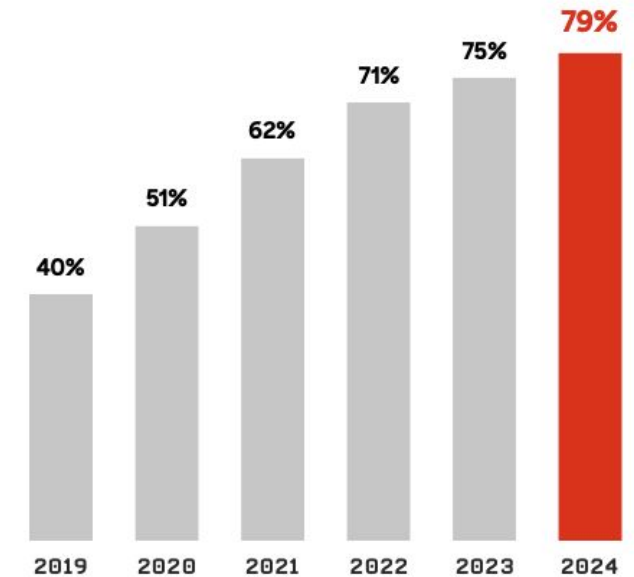


Figure 4. Percentage of detections that were malware-free, 2019-2024

Identity Protection: Schutz Mechanismen

- Kontext Betrachtung des Logins und Erzeugen eines Risiko-Werts:
 - Timing / Frequenz
 - Ort (und Veränderung über Zeit)
 - Geräte & Fingerprints
 - Netzwerk & IP
 - Verhalten
 - Credentials geleakt (Darknet, have i been pwned)
 - Historie / Audits & Meldungen von Integrationen (XDR, SIEM)
- Bedingte / Adaptive Reaktionen auf Risiko:
 - Blockieren
 - MFA
 - Isolation (mit Identity Proxy)
 - Kontensperrung
 - Require Managed Device

Strategie

Infrastruktur smart vernetzen statt Feature-Zoo

- Architektur statt Einzellösungen
 - Technologien (WAAP, EDR, NDR, CNAPP, SIEM/XDR, Mail-GW ...) gezielt kombinieren, klare Rollen pro Komponente.
- Überlappende Features vermeiden
 - Threat Detection, Logging, Sandbox, Automation etc. nur dort einkaufen, wo sie wirklich gebraucht werden → weniger Lizenzkosten.
- Schnittstellen & Datenflüsse planen
 - Wer sammelt welche Logs? Wo wird korreliert? Welche Events gehen wohin? → weniger Komplexität, schnellere Incidents.
- Standardisierung & Konsolidierung
 - Weniger Produkte, dafür tiefe Integration und saubere Betriebsprozesse (Runbooks, Playbooks, Monitoring).
- Erfahrungswissen & Skills entscheidend
 - Architekten müssen Technologien, Limits und Overlaps kennen – nur so entstehen sinnvolle Designs, die Geld sparen statt verbrennen.

